



Faktorinvestering

Prediktera framtida aktiepriser med hjälp av linjär regression

Factor Investing

Kandidatarbete inom civilingenjörsutbildningen vid Chalmers

Erik Jörgensen Broms

Mike Fakhoury

Karl Fredriksson

Eric Glisic

Anton Räfling

Faktorinvestering

Prediktera framtida aktiepriser med hjälp av linjär regression

Kandidatarbete i matematik inom civilingenjörsprogrammet Automation och mekatronik vid Chalmers

Mike Fakhoury Karl Fredriksson

Kandidatarbete i matematik inom civilingenjörsprogrammet Teknisk matematik vid Chalmers

Erik Jørgensen Broms Eric Glisic Anton Räfning

Handledare: Carl Lindberg

Institutionen för Matematiska vetenskaper
CHALMERS TEKNISKA HÖGSKOLA
GÖTEBORGS UNIVERSITET
Göteborg, Sverige 2026

Förord

Detta kandidatarbete har genomförts vid institutionen för Matematiska vetenskaper vid Chalmers tekniska högskola under vårterminen 2026. Skribenterna tillhörde civilingenjörsprogrammen Automation och mekatronik respektive Teknisk matematik.

Arbetet har varit en kollaborativ insats där alla gruppmedlemmar aktivt bidragit till samtliga delar av projektet, med viss uppdelning vad beträffar praktisk kodning respektive skrivarbete. Den praktiska kodimplementationen av modellerna och datainsamlingen genomfördes gemensamt genom parprogrammering och gruppträffar. Den teoretiska bakgrunden för arbetet samt metodbeskrivningen utarbetades i nära samarbete mellan samtliga författare, och rapportdokumentet har i sin helhet reviderats av flera gruppmedlemmar.

Specifika ansvarsområden fördelades enligt följande:

Erik Jörgensen Broms och Eric Glisic ansvarade primärt för den matematiska härledningen av portföljoptimeringen och dess dokumentation. Mike Fakhoury och Karl Fredriksson fokuserade på implementeringen av Fama-MacBeth-regressionen och Anton Råfling utvecklade omfattande signal-diagnostiksfunktionalitet för prestationsutvärdering av olika modelliterationer och ett gemensamt ramverk för modellerna. Resultat- och diskussionsavsnitten författades kollaborativt med jämn arbetsfördelning mellan samtliga gruppmedlemmar.

Under arbetets gång har en detaljerad loggbok förts över gruppens gemensamma arbete samt individuella tidsloggar för varje gruppmedlem. Dessa dokument har kontinuerligt uppdaterats och finns tillgängliga för verifiering.

Vi vill rikta ett stort tack till vår handledare Carl Lindberg för värdefull vägledning, god kompetens och nyttig feedback under arbetets gång.

Göteborg, maj 2026

Anton Råfling, Eric Glisic, Erik Jörgensen Broms, Karl Fredriksson, Mike Fakhoury

Populärvetenskaplig presentation

Kan matematik hjälpa dig att investera smartare?

När de flesta tänker på aktiemarknaden tänker de på Wall Street, börshallar och stressade mäklare. Men bakom kulisserna till denna fartfyllda föreställning arbetar matematiker och ingenjörer med något helt annat: att hitta mönster i marknaden med hjälp av statistik och optimering.

Detta arbete handlar om så kallad faktorinvestering – en metod där man använder företags finansiella nyckeltal för att förutspå vilka aktier som kommer att gå bra. Istället för att gissa eller följa magkänslan bygger vi en matematisk modell som analyserar hundratals företag samtidigt och letar efter gemensamma egenskaper hos framgångsrika aktier.

Tänk dig att du ska skapa din portfölj och väljer bland tusen olika aktier. Hur vet du vilka som är bra köp? Ett sätt är att titta på finansiella nyckeltal – mått som beskriver företags ekonomiska hälsa. Till exempel: Hur lönsamt är företaget? Hur snabbt växer det? Hur är det värderat i förhållande till sina konkurrenter? Dessa mått kallar vi för faktorer.

Vår modell samlar in data om dessa faktorer för hundratals företag över många år. Sedan använder vi linjär regression – en statistisk metod – för att upptäcka vilka faktorer som historiskt sett har varit bäst på att förutspå framgångsrika aktier. Vi fann till exempel att tillgångar som växer måttligt och företag med hög avkastning på investerat kapital tenderar att prestera bättre än marknaden.

Men det räcker inte att bara veta vilka aktier som är bra, man måste också bestämma hur pengar man ska lägga på varje aktie. Det är här portföljoptimeringen kommer in. Med en matematisk metod utvecklad av Harry Markowitz balanserar vi förväntad avkastning mot risk. Målet är att få så hög avkastning som möjligt utan att ta onödigt stora risker.

Våra resultat visar att modellen kan generera cirka 16 % årlig avkastning – något bättre än marknadens 15.5 % – med liknande risk. Det låter kanske inte som en enorm skillnad, men över tid kan dessa extra procentenheter innebära betydande värdeökning. Viktigast är kanske att modellen gör detta systematiskt och objektivt, utan att påverkas av känslor eller tillfälliga trender.

Faktorinvestering används idag av många stora investeringsfonder världen över, och intresset fortsätter att växa. I takt med att datorkraft och tillgång till finansiell data ökar, blir dessa metoder allt mer sofistikerade. Detta arbete visar att även relativt enkla statistiska metoder kan användas för att navigera den komplexa aktiemarknaden.

Sammandrag

Detta kandidatarbete undersöker tillämpningen av faktorbaserad portföljoptimering för att prediktera framtida aktieavkastningar på den nordamerikanska marknaden. Studien använder Fama-MacBeth-regression för att skatta faktorpremier baserade på åtta finansiella nyckeltal inom kategorierna value, growth och performance. Förväntade avkastningar och kovariansmatriser konstrueras utifrån en faktormodell och används därefter i Markowitz medelvariansoptimering för att generera månadsvis ombalanserade portföljer.

Modellen utvärderas genom backtesting över perioden 2019–2025 på ett universum av cirka 1000 aktier. Två parallella strategier jämförs: en aggregerad modell där alla tillgångar optimeras gemensamt (Full-Universe) och en sektorvis modell där Markowitz-optimering genomförs separat för tio marknadssektorer. Prestationen mäts mot S&P 500 som benchmark. Resultaten visar att Full-Universe-modellen uppnår en kumulativ avkastning på 140.09 % motsvarande 15.95 % annualiserad avkastning, vilket marginellt överträffar S&P 500:s 15.5 %. Sharpe-kvoten för Full-Universe är 0.87 jämfört med S&P 500:s 0.75, vilket indikerar högre riskjusterad avkastning. Sektormodellen genererar 14.09 % annualiserad avkastning men med betydligt högre volatilitet (23.27 %) och lägre Sharpe-kvot (0.68), vilket indikerar att sektorvis optimering introducerar instabilitet genom det dubbla skattningsproblemet. Faktoranalys identifierar Asset Growth, ROIC och P/FCF som de statistiskt mest signifikanta prediktorerna med t-statistikor över 1.96. Asset Growth uppvisar starkast signalstyrka ($SNR = 0.277$) i linje med etablerad finansiell teori om investeringsanomalier. Hyperparameteranalys visar att riskversionsparametern θ optimeras kring 4–5, medan EMA-spannet föredrar värden över 40 dagar för att filtrera brus i fundamentala nyckeltal.

Studien indikerar att faktorbaserade strategier *kan* generera modest överavkastning relativt marknadsindex när de implementeras med robust skattningsmetodik och lämplig regularisering av kovariansmatrisen, trots relativt små marginaler mot index under 2019–2025. Resultaten understryker vikten av aggregerad optimering framför sektorvis uppdelning när faktorsignaler är svaga eller instabila inom enskilda sektorer.

Nyckelord: Faktorinvestering, Markowitz-optimering, Fama-MacBeth-regression, portföljoptimering, linjär regression, finansiella faktorer.

Abstract

This bachelor's thesis examines how well factor-based portfolio optimization-models can predict future stock prices in North American stock markets. The study leverages Fama-MacBeth-regression to estimate factor premiums based on eight financial key performance indicators, categorized by value, growth and performance. Expected stock returns and covariance-matrices are constructed from a factor model and are thereafter used in Markowitz Mean-Variance-Optimization to generate portfolios which are re-balanced monthly.

The model is evaluated by out-of-sample backtesting over the period 2019–2025 on a market-universe of about 1000 stocks. Two parallel strategies were compared: an aggregated model where all instruments are jointly optimized (Full-Universe) and a sector-separated model where Markowitz-optimization is conducted separately for ten market sectors. The performance of both models is measured against the S&P 500 as a benchmark metric of market movement. Results show that the Full-Universe model achieves a cumulative return of 140.09 %, equating to a 15.95 % annualized return over the backtesting period. This beats the S&P 500's 15.5 % annualized return over the same period, albeit by a small margin. The Sharpe ratio of the Full-Universe is 0.87 relative to benchmark's 0.75, which indicates that our model produces a higher risk-adjusted return relative to benchmark. The sector-wise model generates a 14.09% annualized return with a high annualized volatility and correspondingly lower Sharpe ratio (0.68) which hints that sector-wise optimization introduces instability by more estimations. Factor analysis identifies Asset Growth, ROIC and P/FCF as the most statistically significant price-predictors with t-statistics above 1.96. Asset Growth has the best signal-strength ($SNR = 0.277$) which aligns with well-established financial theory on investment anomalies. Hyper-parameter analysis shows that the risk-aversion parameter θ has an optimal value in the range (4, 5), while the so-called EMA-span parameter yields better returns if assigned a value larger than 40 days.

The study confirms that factor-based strategies *can* generate a modest surplus return relative to market indices when implemented by robust estimation methods and suitable regularization of the covariance matrix, in spite of small return margins over the backtesting period. Results underline the importance of aggregated optimization as opposed to a divided sector approach when factor signals are weak or otherwise unstable within individual sectors.

Keywords: factor investing, Markowitz-Optimization, Fama-MacBeth-regression, portfolio optimization, linear regression, financial factors.

Innehåll

Begreppslista	i
1 Introduktion	1
1.1 Syfte	1
1.2 Avgränsningar	1
1.3 Samhällsenliga och etiska aspekter	1
1.4 Frågeställning	2
2 Teori	2
2.1 Markowitzramverket för portföljoptimering	2
2.2 En optimal portföljstrategi	2
2.3 Praktiska problem vid hög dimensionalitet	3
2.4 Faktormodeller	4
2.5 Faktorinvestering	4
2.6 Shrinkage-metoden	4
3 Metod	5
3.1 Datakälla och datastruktur	5
3.2 Datarensning och förbehandling	5
3.3 Faktorbearbetning	6
3.4 Val av faktorer	6
3.5 Fama-MacBeth-regression och μ -skattning	6
3.5.1 Steg 1: Tvärsnitts-OLS per tidpunkt	6
3.5.2 Steg 2: Tidsmedelvärdena $\hat{\lambda}_j$	7
3.5.3 Motivering av Fama-MacBeth-metoden	7
3.6 Konstruktion av kovariansmatrisen	7
3.7 Portföljoptimering	8
3.7.1 Modellvarianter	8
3.8 Back-Testing och prestationsutvärdering	8
3.9 Sektormodell	9
4 Resultat	9
4.1 Backtestperiod och inställningar	9
4.2 Faktoruppsättning	9
4.3 Portföljprestanda: Full-Universe vs. Sektor vs. Benchmark	10
4.4 Faktorerers prediktionsförmåga och Sharpe-kvotens beroende på faktoruppsättning	11
4.5 Riskaversionens påverkan	13
5 Diskussion	13
5.1 Besvarar modellen studiens frågeställningar?	13
5.1.1 Fråga 1: Vilken annualiserad avkastning kan uppnås?	13
5.1.2 Fråga 2: Vilka faktorer har bäst prediktiv förmåga och hur påverkar faktoruppsättning Sharp-kvoten?	14
5.1.3 Fråga 3: Hur påverkar hyperparametrar modellens prestanda?	16
5.2 Slutsats	17
AI-användande	I
A Kompletterande Teori	I
A.1 Z-score	I
A.2 Rang-standardisering	I
A.3 Exponentiell viktning (EMA)	I
A.4 Kostnad för handel	I

B Utökade resultat	II
B.1 Sektorvikter	II
C Matematiska bevis	II
C.1 Bevis optimal portfölj	II
D Källkod	III

Begreppslista

CAPM	<i>Capital Asset Pricing Model</i> – Klassisk modell för prissättning av finansiella tillgångar. Hävdar att förväntad avkastning enbart förklaras av exponering mot marknadsrisk (beta).
Beta (β)	Mått på en tillgångs systematiska risk relativt marknaden. $\beta > 1$ innebär högre rörlighet än marknaden; $\beta < 1$ innebär lägre risk.
Faktorinvestering	Investeringsstrategi som identifierar och utnyttjar observerbara egenskaper (faktorer) hos aktier för att prediktera framtida avkastning.
Faktor	Mätbar egenskap hos ett finansiellt instrument t.ex. P/E-tal, momentum eller rörelsemarginal som används för att förklara tvärsnittlig aktieavkastning.
Faktorpremie (λ_j)	Den genomsnittliga ersättning marknaden ger för exponering mot en viss faktor, skattad via Fama–MacBeth-regression.
Fama–MacBeth-regression	Twåstegsmetod för att skatta faktorpremier: (1) tvärsnittsregression per tidpunkt, (2) tidsmedelvärdena av de skattade koefficienterna testas statistiskt.
Kovariansmatris (C)	$n \times n$ -matris vars element C_{ij} anger kovariansen mellan avkastningarna för tillgång i och j . Central i Markowitz-optimering.
Markowitz-optimering	Portföljoptimeringsmetod som maximerar förväntad avkastning givet en risknivå (eller minimerar risk givet avkastning). Löser $\max_{\pi} \{E[R^{\pi}] - \theta \cdot \text{Var}(R^{\pi})\}$.
Optimal portföljstrategi (π^*)	Den viktvektor som löser Markowitz-optimeringsproblemet: $\pi^* = C^{-1} \cdot \frac{1}{2\theta}(\mu - r\mathbf{1}).$
Portföljvikt (π_i)	Andelen av det totala investerade kapitalet som placeras i tillgång i .
Riskavversionsparameter (θ)	Styr avvägningen mellan förväntad avkastning och risk i målfunktionen. Högt θ ger mindre risk; lågt θ ger mer risk.
Risikfri ränta (r)	Avkastningen på en risikfri placering (t.ex. statsskuldväxlar), som utgör referensnivå i Markowitz-ramverket.
Shrinkage (Ledoit–Wolf)	Metod för att stabilisera kovariansmatrisen genom att kombinera den sampelade matrisen med identitetsmatrisen: $C_{\text{shrink}} = \rho C + \sigma^2(1 - \rho)I$.
Faktormodell	Modell som antar att avkastningen för tillgång i drivs av ett antal gemensamma faktorer $Y_1, \dots, Y_q$ plus en tillgångsspecifik brusterm.
Faktorexponeringsmatris (B)	$n \times q$ -matris vars rad i innehåller tillgång i :s rang-standardiserade faktörvärden $\hat{F}_{i,j}$.
Idiosynkratisk risk ($\sigma_{\epsilon,i}^2$)	Den tillgångsspecifika variansen som ej förklaras av de gemensamma faktorerna; skattad från Fama–MacBeth-residualerna.

Informationskoefficient (IC)	Spearman's rangkorrelation mellan faktorvärden och realiserade framtida avkastningar vid en given tidpunkt. Mäter faktorns prediktiva förmåga.
Signal-to-Noise Ratio (SNR)	Absolutvärdet av medelvärde faktorpremie dividerat med dess standardavvikelse över tid. $SNR > 2$ indikerar en signal urskiljbar från brus.
Look-ahead bias	Systematiskt fel som uppstår när modellen vid tidpunkt t oavsiktligt använder information som först är tillgänglig vid $t' > t$.
Winsorisering	Statistisk metod för hantering av extremvärden: värden utanför en given percentilgräns ersätts med själva percentilvärdet snarare än att tas bort.
Rangstandardisering	Transformation av faktorvärden till intervallet $[-1, 1]$ baserat på tvärsnittsrankning vid varje tidpunkt, för att göra faktorer jämförbara över tid och instrument.
EMA (Exponentiellt glidande medelvärde)	Viktningsschema som ger nyare observationer exponentiellt högre vikt: $EMA_t = \alpha F_{i,t} + (1 - \alpha) EMA_{t-1}$.
Z-score	Standardiseringsmått: $Z = (x - \mu)/\sigma$. Anger hur många standardavvikelser ett värde ligger från medelvärdet.
Sharpe-kvot	Överskottsavkastning relativt riskfri ränta, dividerad med annualiserad volatilitet. Mäter riskjusterad avkastning.
Maximum Drawdown	Den maximala kumulativa förlusten från en lokal topp till efterföljande botten.
Information Ratio (IR)	Genomsnittlig aktiv avkastning (portfölj minus benchmark) dividerad med tracking error.
Tracking Error	Standardavvikelsen för den aktiva avkastningen (portfölj minus benchmark), annualiserad.
Backtest	Historisk simulering av portföljstrategi på faktisk marknadsdata för att utvärdera modellens prestanda.
Burn-in-period	Inledande period under vilken data samlas och modellen tränas, men ingen portföljkonstruktion eller handel sker.
OHLCV	Förkortning för <i>Open, High, Low, Close, Volume</i> . Standardformat för daglig kursdata.
R12 (rullande 12 månader)	Rapporteringsformat där nyckeltal beräknas som summan av de senaste fyra kvartalen, vilket eliminerar säsongseffekter.
P/E-tal	Pris dividerat med vinst per aktie. Klassisk värderingsfaktor.
P/FCF	Pris dividerat med fritt kassaflöde per aktie.
EV/EBIT	Företagsvärde dividerat med rörelseresultat. Värderingsmultipel.
ROIC	<i>Return on Invested Capital</i> . Avkastning på investerat kapital.
OCF Margin	Operativ kassaflödesmarginal. Operativt kassaflöde som andel av omsättningen.

Gross Margin	Bruttomarginal. Nettoomsättning minus kostnad för sålda varor, dividerat med nettoomsättningen. Mäter hur stor andel av intäkterna som återstår efter direkta produktionskostnader.
Momentum	Faktor baserad på historisk kursutveckling, typiskt de senaste 12 månaderna, som mäter prisrörelsetrender.
Asset Growth	Tillgångstillväxt. Förändring i totala tillgångar över tid. En av de starkaste faktorerna i studien.
ESG	<i>Environmental, Social and Governance</i> Ramverk för att bedöma bolags hållbarhet och etik i investeringssammanhang.

1 Introduktion

Modern portföljteori grundades av Harry Markowitz på 1950-talet och bygger på idén att en investerare bör välja portföljer genom att väga förväntad avkastning mot risk, mätt som varians i avkastning. Detta leder klassiskt till ett optimeringsproblem där portföljvikterna, de andelar av det totala investerade kapitalet som placeras i varje given aktie, bestäms dels av aktiernas förväntade avkastningar, dels av den så kallade kovariansmatrisen mellan deras avkastningar. Detta brukar kallas medelvariansoptimering av aktieportföljer (Markowitz, 1952).

Faktorinvestering är ett område som befinner sig i gränslandet mellan ovannämnda ekonomi, statistik och beteendevetenskap. En av de mest grundläggande frågorna inom finansiell teori är vad som bestämmer den förväntade avkastningen på riskfyllda tillgångar. Den klassiska kapitalprissättningsmodellen (CAPM) erbjuder ett elegant och intuitivt svar: skillnader i förväntad avkastning förklaras enbart av tillgångarnas exponering mot marknadsrisk. I modellen är det endast den systematiska risken, mätt genom så kallat beta, som belönas med en riskpremie, medan idiosynkratisk risk inte belönas.

Forskning har dock visat att CAPM inte förmår förklara flertalet robusta mönster i data (Fama och French, 1992). Aktier med liknande marknadsbeta kan uppvisa stora skillnader i avkastning över tid. Sådana observationer indikerar att ytterligare riskdimensioner eller alternativa mekanismer kan spela en viktig roll i bestämmandet av ett värdepappers genomsnittliga avkastning. Faktorinvestering växer fram ur denna diskrepans mellan teori och praktik och kan ses som ett försök att identifiera och modellera de variabler som bättre förklarar tvärsnittet av aktieavkastningar.

1.1 Syfte

Syftet med arbetet är att undersöka om faktorbaserad portföljoptimering kan generera riskjusterad överavkastning relativt marknadsindex på den nordamerikanska aktiemarknaden. Detta görs genom att kombinera Fama-MacBeth-regression för skattning av förväntade avkastningar med Markowitz-medelvariansoptimering för portföljkonstruktion. Arbetet utvärderar hur modellernas prestation påverkas av faktorval, sektorindelning och riskavsnivå, och drar slutsatser om vilka faktorer som bäst fångar underliggande avkastningsmönster i data.

1.2 Avgränsningar

Arbetet begränsas till att behandla cirka 1000 aktier från de största indexen på den nordamerikanska marknaden. Övriga marknader – däribland asiatiska och nordiska börser samt tillväxtmarknader – inkluderas inte, huvudsakligen av datatekniska skäl samt för att hålla analysens omfång inom ramen för ett kandidatarbete.

Vad gäller valet av faktorer behandlar arbetet åtta av de vanligast förekommande finansiella faktorerna. Det finns faktorer med stark förklaringskraft som inte berörs här, men att identifiera eller konstruera dem är ett arbete i sig.

1.3 Samhällsenliga och etiska aspekter

Faktorbaserad aktieportföljoptimering bygger på kvantitativa modeller som fokuserar på finansiella nyckeltal och historiska prisrörelser, utan hänsyn till om de bolag som investeras i uppfyller etiska eller hållbara standarder. I praktiken kan modellen välja att investera i företag i kontroversiella branscher, med hög klimatpåverkan eller med bristande arbetsmiljöstandard. Den kvantitativa modellen saknar förmåga att bedöma dessa dimensioner, eftersom de sällan uttrycks i de nyckeltal som används. Finansiell prestation och etisk prestation är inte nödvändigtvis korrelerade. Investerare som önskar integrera etisk hänsyn kan komplettera modellen med negativ screening — exklusion av specifika branscher — eller positiv screening via så kallade ESG-betyg från exempelvis MSCI eller Sustainalytics. Ansvaret för etiska investeringsbeslut ligger slutligen hos den enskilda investeraren.

1.4 Frågeställning

1. Vilken annualiserad avkastning kan en faktorbaserad portföljmodell uppnå på den nordamerikanska marknaden, och hur presterar denna relativt ett marknadsindex?
2. Vilka finansiella faktorer har bäst prediktiv förmåga i bemärkelsen aktieavkastningar och hur varierar denna förklaringskraft mellan olika sektorer? Hur påverkar antalet faktorer i kombination den riskjusterade avkastningen?
3. Hur påverkar riskaversionsgrad annualiserad avkastning och modellprestation?

2 Teori

I detta avsnitt presenteras den matematiska och finansiella teorin som utgör grunden för arbetet. Inledningsvis presenteras Markowitzramverket för portföljoptimering, en härledning av den optimala strategin, och därefter en diskussion kring de praktiska problem som uppstår vid tillämpning av detta ramverk på marknader med många tillgångar. Mot slutet introduceras faktormodeller och faktorinvestering som lösning på detta problem samt shrinkage-metoden för numerisk stabilitet.

2.1 Markowitzramverket för portföljoptimering

Betrakta en marknad med n riskfyllda tillgångar samt ett penningmarknadskonto via vilket investerare kan allokera kapital bland dessa tillgångar. Investeraren befinner sig vid tidpunkt $t = 0$ och väljer en *portföljstrategi* $\boldsymbol{\pi} = (\pi_1, \pi_2, \dots, \pi_n)$ där π_i avser andelen av allt tillgängligt kapital som investeraren placerar i tillgång i . **Avkastningen** för tillgång i definieras enligt:

$$R_i = \frac{S_i(1) - S_i(0)}{S_i(0)} \quad (1)$$

där $S_i(t)$ betecknar priset på tillgång i vid tidpunkten t . Avkastningar för samtliga tillgångar vid en given tid samordnas enligt $\mathbf{R}_t = (R_1, R_2, \dots, R_n)$. Vidare antas penningmarknadskontot ge den riskfria räntan r , det vill säga $R_{n+1} = r$. Total portföljavkastning vid en tid t ges då av

$$R^\pi := \mathbf{R}_t \cdot \boldsymbol{\pi} = \sum_{i=1}^{n+1} \pi_i R_i \quad (2)$$

under antagandet att allt kapital är investerat, $\sum_{i=1}^{n+1} \pi_i = 1$. Med (1) och (2) formulerar Markowitz optimeringsproblemet att maximera den förväntade avkastningen med en straffterm för risk:

$$\max_{\boldsymbol{\pi}} \{ \mathbb{E}[R^\pi] - \theta \cdot \text{Var}(R^\pi) \}. \quad (3)$$

Här är θ en *riskaversionsparameter*: ett lägre värde motsvarar en riskbenägen investeringsstrategi, medan ett högre värde korresponderar med en lägre grad av risktagande.

2.2 En optimal portföljstrategi

Målfunktionen (3) kan bearbetas vidare. Den totala förväntade avkastningen för portföljen $\mathbb{E}[R^\pi]$ kan uttryckas som

$$\mathbb{E}[R^\pi] = \sum_{i=1}^{n+1} \pi_i \mathbb{E}[R_i] = \sum_{i=1}^{n+1} \pi_i \mu_i$$

där $\mu_i := \mathbb{E}[R_i]$ betecknar förväntad avkastning för tillgång i och $\mu_{n+1} = r$. Variansen i portföljavkastning $\text{Var}(R^\pi)$ beräknas via definitionen för varians och ger då:

$$\text{Var}(R^\pi) = \mathbb{E}[(R^\pi - \mathbb{E}[R^\pi])^2] = \sum_{i,j=1}^n \pi_i \pi_j \mathbb{E}[(R_i - \mu_i)(R_j - \mu_j)]$$

och summationsindex i går upp till n snarare än $n + 1$, då variansen för penningmarknadskontot är noll. Vidare noteras att uttrycket

$$\mathbb{E}[(R_i - \mu_i)(R_j - \mu_j)]$$

är kovariansen c_{ij} mellan avkastningarna för tillgångarna i och j . Låt C beteckna *kovariansmatrisen* med element:

$$C_{i,j} = c_{ij} \quad \forall i, j = 1, \dots, n \quad (4)$$

Eftersom bivillkoret kräver att $\pi_{n+1} = 1 - \sum_{i=1}^n \pi_i$, kan denna variabel elimineras varpå (3) omformuleras enbart i termer av de n riskfyllda tillgångarna:

$$g(\pi_1, \dots, \pi_n) = r + \sum_{i=1}^n \pi_i(\mu_i - r) - \theta \sum_{i,j=1}^n \pi_i \pi_j c_{ij} \quad (5)$$

I syfte att finna ett nödvändigt villkor för optimalitet ansätts $\partial g / \partial \pi_i = 0$ varifrån det erhålls:

$$\frac{\partial g}{\partial \pi_i} = \mu_i - r - 2\theta \sum_{j=1}^n \pi_j c_{ij} = 0 \quad \forall i = 1, \dots, n \quad (6)$$

Då ovanstående måste gälla för samtliga i översätts villkoren till matrisekvationen $C\boldsymbol{\pi} = \frac{1}{2\theta} \cdot (\boldsymbol{\mu} - r\mathbf{1})$, där $\boldsymbol{\mu} = (\mu_1, \dots, \mu_n)^T$ och där $\mathbf{1}$ är den n -dimensionella radvektor vars alla element är ett. Under förutsättning att kovariansmatrisen C är inverterbar fås då en kandidat till optimal portföljstrategi:

$$\boldsymbol{\pi}^* = C^{-1} \cdot \frac{1}{2\theta} (\boldsymbol{\mu} - r\mathbf{1}) \quad (7)$$

Det kan visas att $\boldsymbol{\pi}^*$ faktiskt utgör ett globalt maximum för målfunktionen (5); beviset återfinns i appendix C.

2.3 Praktiska problem vid hög dimensionalitet

Som noterat i avsnitt 2.1 har optimeringsproblemet (3) en väldefinierad lösning i teorin. I praktiken uppstår dock allvarliga implementationsproblem vid applikation på verklig marknadsdata. Problematiken härrör i första hand från behovet att skatta parametrar $\boldsymbol{\mu}$ och C från historiska data. En ad-hoc-lösning av detta skattningsproblem kräver oerhörda mängder data, vilket exemplifieras i efterföljande stycken.

Ett 95 %-konfidensintervall för en tillgångs förväntade avkastning ges approximativt av

$$\mu \in (\bar{X}_n - 1.96 \frac{\sigma}{\sqrt{n}}, \bar{X}_n + 1.96 \frac{\sigma}{\sqrt{n}})$$

där $\bar{X}_n$ är stickprovsmedelvärdet av n observationer och σ är standardavvikelsen. För typiska aktier med $\mu \approx 0.08$, $\sigma \approx 0.20$ och en intervallbredd på 0.01, hade konstruktionen av ett sådant krävt approximativt 19 000 år av finansiell data. Att öka samplingsfrekvensen förbättrar inte heller precisionen av μ -skattningen.

Det andra problemet uppstår när antalet tillgångar n är stort. Kovariansmatrisen C har $O(n^2)$ element som alla måste skattas, och i praktiska tillämpningar är $n > 1000$ inte ovanligt. Att invertera C för stora n blir numeriskt arbetsamt och även instabilt: små, bruskänsliga c_{ij} genererar vid invertering extremt stora c_{ij}^{-1} , vilket föreskriver extrema portföljallokeringar, känsliga för små variationer i den ursprungliga kovariansmatrisen.

För att hantera dessa skattningsproblem använder denna studie två komplementära strategier: (1) faktormodeller för att reducera antalet fria parametrar från $O(n^2)$ till $O(n \cdot q)$, beskrivet i avsnitt 2.4, och (2) Ledoit-Wolf shrinkage för numerisk stabilisering av kovariansmatrisen, beskrivet i avsnitt 2.6.

2.4 Faktormodeller

Ett vanligt tillvägagångssätt för att hantera problemet med höga dimensioner är att strukturera beroendet mellan tillgångarnas avkastningar med hjälp av ett litet antal gemensamma faktorer. En faktor avser en mätbar egenskap hos ett finansiellt instrument, som exempelvis finansiella nyckeltal eller sammanslagningar av dessa. I en statistisk faktormodell antas att avkastningen för tillgång i kan skrivas som

$$R_i = \mu_i + \sigma_{i_0}X_i + \sum_{j=1}^q \sigma_{i,j}Y_j \quad (8)$$

där X_i är en tillgångsspecifik brusterm, $Y_1, \dots, Y_q$ är gemensamma systematiska faktorer med noll-medelvärde samt enhetsvarianser och $\sigma_{i,j}$ är exponeringsfaktorer som mäter tillgång i 's exponering mot faktor j (Fama och French, 1993). Det antas vidare att $[X_i]_{i=1}^n$ är oberoende, dels av varandra, dels av faktorerna. Kovariansen mellan tillgång i och j ges under denna modell av:

$$c_{ij} = \text{Cov}(R_i, R_j) = \sum_{k=1}^q \sigma_{i,k}\sigma_{j,k} \quad (9)$$

Med (8) och (9) behöver man nu endast skatta $O(n \cdot q)$ faktorladdningar $\sigma_{i,j}$, vilket är hanterbart så länge q är litet. Faktorerna definieras i detta arbete a priori utifrån finansiella nyckeltal.

2.5 Faktorinvestering

Faktorinvestering avser användandet av faktormodeller för att prediktera framtida avkastningar μ_i och utifrån detta konstruera en portfölj. Inom faktorinvestering antas det att den förväntade avkastningen för tillgång i kan skrivas som en linjär funktion av ett antal observerbara faktorer $F_1, \dots, F_m$ för något $m \in \mathbb{N}$:

$$\mu_i := E[R_i] = \sum_{j=1}^m \lambda_j F_j \quad (10)$$

där faktorpremierna $[\lambda_j]_{j=1}^m$ är koefficienter som skattas med ordinär linjär regression (OLS).

Exempel på vanligt förekommande faktorer är pris-till-vinstkvoten (P/E), rörelsemarginal, skuldsättningsgrad samt olika momentumvariabler baserade på historisk kursutveckling. Valet av faktorer är en central modelleringsfråga eftersom olika faktorer fångar olika dimensioner av börsnoterade bolags fundamentala egenskaper. Vilka faktorer som bäst förklarar framtida avkastning i den aktuella marknaden är en intressant empirisk fråga.

Med skattade μ_i från faktormodellen (se avsnitt 3.5) kan den optimala Markowitzportföljen π^* konstrueras för varje tidsperiod. Portföljens kumulativa avkastning över N perioder ges av:

$$\prod_{i=1}^N (1 + R_t^{\pi}) \quad (11)$$

2.6 Shrinkage-metoden

Som nämnt i avsnitt 2.3 blir invertering av kovariansmatrisen problematiskt eftersom små egenvärden exploderar till enorma tal. Ett sätt att hantera denna numeriska instabilitet är att ersätta den skattade kovariansmatrisen med en linjärkombination av sig själv och identitetsmatrisen.

$$C_{shrink} = \rho C + \sigma^2(1 - \rho)I, \quad \rho \in [0, 1] \quad (12)$$

Här styr parametern ρ graden av stabilisering mot identitetsmatrisen. När $\rho = 1$ används den ursprungliga skattningen och när $\rho = 0$ ersätts kovariansmatrisen av $\sigma^2 I$, vilket innebär att alla tillgångar antas vara oberoende med lika varianser. Valet av ρ sker typiskt via **Ledoit-Wolf's** formel (Ledoit och Wolf, 2004). Effekten av shrinkage är att extrema egenvärden i den skattade

kovariansmatrisen dämpas, vilket stabiliserar inversen och förhindrar extrema portföljvikter.

Den statistiska motiveringen till **Ledoit-Wolf Shrinkage** bygger på det så kallade Stein-fenomenet: i högdimensionella skattningsproblem kan man visa att det finns skattare med lägre kvadratisk medelfel än maximum-likelihood-skattaren (Ledoit & Wolf, 2004). Shrinkage-matrisen kan ses som en regularisering som introducerar ett kontrollerat bias mot en enklare, mer stabil modell av oberoende tillgångar. Detta i utbyte mot en avsevärt minskad varians i skattningen av de portföljvikter som härleds ur kovariansmatrisen.

Det bör noteras att shrinkage-metoden förhindrar katastrofala portföljkonfigurationer men ger inte nödvändigtvis den teoretiskt korrekta modellen. Det finns ingen garanti för att den krympta matrisen bättre återspeglar den sanna kovariansstrukturen på marknaden utan shrinkage är snarare en pragmatisk lösning. I avsnitt 3.6–3.7 beskrivs hur metoden används i detta arbete.

3 Metod

I följande avsnitt kommer metoden att beskrivas. Metoden har sin utgångspunkt i datainsamling och löper sedan genom databehandling, faktorbearbetning, skattning av förväntad avkastning, portföljkonstruktion och slutligen utvärdering.

3.1 Datakälla och datastruktur

All fundamental data och historiska kurser hämtades via Börsdatas API (borsdata.se). Börsdata tillhandahåller strukturerade finansiella data för den amerikanska marknaden, inklusive historiska nyckeltal (KPI:er) på instrument- och periodnivå. För att undvika look-ahead-bias eftersträvades ett urval av bolag som inte enbart baserades på dagens marknadsvärde eller framgång. Bland valda aktier fanns stor variation i nutida prestanda. I syfte att motverka *Survivorship Bias* - ett förvrängt och partiskt investeringsförfarande där modellen omedvetet endast investerar i historiskt solida bolag med stark kursgång - inkluderades även aktier med undermåligare prestationsperioder eller bolag som idag försvunnit från index.

Fundamentala nyckeltal hämtades med rapporteringstypen R12, vilket innebär rullande tolv månader. Formatet tar bort säsongeffekter som annars kan ge snedvridna bilder av enskilda kvartalsprestationer, samtidigt som det undviker diskreta diskontinuiteter i data som återfinns i traditionella årsredovisningar. Dessutom gör R12-formatet det möjligt att jämföra bolags nyckeltal även då de har olika räkenskapsår. Kursdatan hämtades i omgångar om maximalt 50 instrument åt gången, i enlighet med API:ns begränsning.

Kursdatan hämtades som dagliga OHLCV-observationer (öppning, high, low, stängning, volym) för det valda datumintervallet. De två datatyperna – kurser och nyckeltal – sammanfogades sedan till en gemensam datastruktur. Detta är nödvändigt för att undvika look-ahead bias: att modellen vid tidpunkt t använder information som först blir tillgänglig vid $t' > t$.

3.2 Datarensning och förbehandling

Rådata genomgick en strukturerad process där den först sorterades per instrument och datum för att sedan rensas från dubletter. Därefter tillämpades fyra bearbetningssteg med direkt inverkan på modellens stabilitet:

Forward-fill av fundamentala nyckeltal: eftersom nyckeltal uppdateras kvartals- eller årsvis hölls det senast rapporterade värdet konstant fram till nästa rapport. För att undvika look-ahead bias relaterad till försenad rapportering försköts data med en tidsperiod på två månader. Linjär interpolation undveks eftersom den skulle introducera framtida information i historiska observationer.

Winsorisering tillämpades på både faktorer och avkastningsserien vid den 1:a och 99:e percentilen för att förhindra att extremvärden — exempelvis kraftigt negativt P/E vid förlustår — dominerar regressionen och snedvrider λ -skattningarna (Lien & Balakrishnan, 2023).

Z-score-standardisering tillämpades på faktornvärdena för att göra dem jämförbara oavsett ursprunglig skala och för att förhindra att faktorer med stora absoluta värden dominerar regressionen. Varje faktornvärde uttrycks därmed i antal standardavvikelser från tvärsnittsnittet vid tidpunkt t (Milton & Arnold, 2003). För en djupare teoretisk förklaring, se Appendix A.1.

Logaritmtransformering med $\log(1 + x)$ tillämpades på börsvärde och handelsvolym, vars fördelningar är kraftigt högerskeva med värden över flera storleksordningar (West, 2022).

3.3 Faktorbearbetning

Innan faktornvärdena används i regressionen genomgick de s.k. rang-standardisering och exponentiell viktning.

Rang-standardisering tillämpades för att minska känsligheten för extremvärden samt för att göra faktorer jämförbara över tid och mellan instrument (Guo m.fl., 2022, p. 5). Mer precis sorterades värdena för en given faktor F_j över alla tillgångar i vid tidpunkt t , varefter varje observation tilldelades en rang i intervallet $[-1, 1]$, vilket möjliggjorde jämförbarhet. För en fullständig beskrivning se Appendix A.2.

Exponentiell viktning (EMA) tillämpades på de rang-standardiserade faktornvärdena för att dämpa kortsiktigt brus i de fundamentala nyckeltalen och samtidigt ge nyare observationer större vikt (Morales-Brotons m.fl., 2024). Viktningen beror dessutom på parametern *span* som styr modellens minneslängd och dess inverkan på modellprestanda undersöks i avsnitt 3.8. För den intresserade finns en mer detaljerad beskrivning av exponentiell viktning i Appendix A.3.

3.4 Val av faktorer

De finansiella nyckeltalen agerar som faktorer som representerar tre olika kategorier: **value**, **growth** och **performance**. Valet baserades på en kombination av väletablerade akademiska resultat. Utgångspunkten var faktoruppsättningen i Fama-French Five Factor Model (Fama och French, 2015), kompletterat med ytterligare faktorer för att få en fullständigare bild av bolags finansiella karaktärstik utefter **value**, **growth** och **performance** (Graham och Dodd, 1934); (Fama och French, 1993). Tillväxtorienterade mått fångar marknadens förväntningar om framtida kassaflöden, medan prestationsbaserade faktorer, såsom avkastning på eget kapital, relaterar till företags operativa effektivitet. Särskilda lönsamhetsmått har visats vara starkt kopplade till framtida avkastning (Novy-Marx, 2013). De åtta faktorer som slutligen valdes listas i avsnitt 4.2.

3.5 Fama-MacBeth-regression och μ -skattning

Den förväntade avkastningen μ_i för instrument i skattades med Fama-MacBeth-regression (Fama & Macbeth, 1973) - ett tvärsnittsförfarande som skattar faktorpremierna $[\lambda]_{i=1}^m$ och testar deras statistiska signifikans.

3.5.1 Steg 1: Tvärsnitts-OLS per tidpunkt

För varje datum t i datamängden genomfördes en OLS-regression med de rang-standardiserade och EMA-viktade faktornvärdena som oberoende variabler och framtida avkastning som beroende variabel:

$$R_{i,t+h} = \lambda_{0,t} + \sum_j \lambda_{j,t} \hat{F}_{i,j,t} + \epsilon_{i,t}$$

där h är prediktionshorisontens längd i antal perioder, $\lambda_{j,t}$ är faktorpremien för faktor j vid tidpunkt t , och $\epsilon_{j,t}$ är residualen. Tidpunkter med färre observationer än antalet faktorer adderar med en konstant exkluderades för att undvika underbestämda ekvationssystem.

3.5.2 Steg 2: Tidsmedelvärdena $\hat{\lambda}_j$

Faktorpremien för faktor j skattades som tidsmedelvärdet av de T tvärsnitts-estimatn:

$$\hat{\lambda}_j = \frac{1}{T} \sum_{t=1}^T \lambda_{j,t}$$

Signifikansen testades med ett t-test där t-statistikan

$$t_j = \frac{\hat{\lambda}_j}{s(\lambda_j)/\sqrt{T}}, \quad \text{med } s(\lambda_j) = \sqrt{\frac{1}{T-1} \sum_{t=1}^T (\lambda_{j,t} - \hat{\lambda}_j)^2}$$

är approximativt normalfördelad under nollhypotesen $\lambda_j = 0$. En faktor anses därmed statistiskt signifikant på 5%-nivån om $|t_j| > 1.96$.

Den förväntade avkastningen för instrument i skattades slutligen som:

$$\hat{\mu}_i = \sum_j \hat{\lambda}_j \hat{F}_{i,j}$$

där $\hat{F}_{i,j}$ är instrumentets senast tillgängliga rang-standardiserade faktorvärde. Residualerna $\epsilon_{i,t}$ sparades per aktie för att senare skatta den idiosynkratiska variansen $\sigma_{\epsilon_i}^2$. De historiska medelvärdena $\hat{\lambda}_j$ används för att skatta förväntad avkastning snarare än enbart de senaste estmaten; implikationerna av detta val diskuteras i avsnitt 5.

3.5.3 Motivering av Fama-MacBeth-metoden

Fama-MacBeth-regression valdes framför alternativa metoder som poolad OLS eller fixed-effects panel regression av tre skäl. För det första undviker metoden problemet med seriell korrelation som uppstår när samma instrument observeras över tid, genom att estimerar koefficienter separat per tidpunkt. För det andra möjliggör tvåstegsförfarandet robusta t-tester av faktorpremiernas statistiska signifikans utan att behöva anta konstanta koefficienter över tid. För det tredje är metoden väletablerad i finansiell forskning (Fama och Macbeth, 1973; Fama och French, 2015), vilket underlättar jämförelse med tidigare studier.

3.6 Konstruktion av kovariansmatrisen

Den faktorbaserade kovariansmatrisen konstruerades utefter Barra-modellen (Grinold & Kahn, 2000, s. 72). Låt B beteckna faktorexponeringsmatrisen av dimension $n \times q$, där rad i innehåller de rang-standardiserade faktorvärdena $\hat{F}_{i,j}$ för instrument i . Låt Σ_F beteckna kovariansmatrisen för faktorpremierna λ_i , skattad från Fama-MacBeth-regressionen. Kovariansmatrisen skattades då som:

$$C_B = B \Sigma_F B^T + D \tag{13}$$

där $D = \text{diag}(\sigma_{\epsilon,1}^2, \dots, \sigma_{\epsilon,n}^2)$ är en diagonalmatrix med de instrumentspecifika idiosynkratiska varianserna från Fama-MacBeth-residualerna. Den första termen $B \Sigma_F B^T$ fångar det systematiska beroendet mellan instrument via de gemensamma riskfaktorerna, medan D fångar den instrumentspecifika risken. Denna konstruktion reducerar antalet fria parametrar från $O(n^2)$ för den samplade kovariansmatrisen till $O(n \cdot q + q^2)$. En ytterligare numerisk regularisering tillämpades genom att addera $10^{-4} \cdot I$ till den konstruerade kovariansmatrisen, vilket syftar till att göra matrisen positivt definit och säkerställa att den numeriska optimeringen konvergerar stabilt.

3.7 Portföljoptimering

Den optimala portföljstrategin beräknades genom att lösa Markowitz mean-variance-optimeringsproblemet (3) under bivillkoret att allt kapital är investerat och att $\pi_i \geq 0 \quad \forall i$. En transaktionskostnad inkluderades i optimeringen för att ta hänsyn till kostnaden av att köpa och sälja finansiella instrument, för mer ingående förklaring av transaktionskostnad se Appendix A.4.

Parametern θ , som styr avvägningen mellan förväntad avkastning och risk, valdes i förväg och flera värden testades för att undersöka dess påverkan på portföljens avkastning och egenskaper.

3.7.1 Modellvarianter

Två parallella portföljmodeller konstruerades för att jämföras med varandra:

1. **Full-universe-modell:** Samtliga aktier optimerades tillsammans i en enda Markowitz-ram utan sektorindelning.
2. **Sektormodell:** Markowitz-optimering sektorvis, följt av en sektorallokering baserad på respektive sektors förväntade avkastning (μ).

Modellerna delade samma faktoruppsättning och hyperparametrar.

3.8 Back-Testing och prestationsutvärdering

Modellen utvärderades genom en historisk backtest. Vid varje ombalanseringstidpunkt – den första handelsdagen i varje månad – beräknades portföljvikterna utifrån all data tillgänglig upp till och med den tidpunkten, varefter portföljens faktiska avkastning under den påföljande månaden beräknades. En burn-in-period på minst 12 månader implementerades för att säkerställa att regressionen har baserats på tillräckligt med data.

En månatlig portföljavkastning och kumulativ avkastning beräknas som i ekvationerna (2) och (11). Följande prestationsmått beräknades och rapporteras för varje modellvariant:

1. **Kumulativ avkastning:** den totala värdeutvecklingen under hela testperioden, $V_T - 1$, där V_T är portföljens ackumulerade procentuella värdeförändring efter T perioder, se (11).
2. **Annualiserad avkastning:** geometrisk medelavkastning per år, beräknad enligt $V_T^{\frac{\text{freq}}{T}} - 1$, där freq avser antalet perioder per år.
3. **Annualiserad volatilitet:** standardavvikelsen för de periodiska avkastningarna skalad med $\sqrt{\text{freq}}$.
4. **Sharpe-kvot:** överskottsavkastning relativt den riskfria räntan dividerad med den annualiserade volatiliteten.
5. **Maximum Drawdown:** den maximala kontinuerliga förlusten från en lokal topp till efterföljande botten i den kumulativa avkastningsserien, ett mått på portföljens worst-case-risk.
6. **Tracking Error:** standardavvikelsen för aktiv avkastning (portfölj minus benchmark), annualiserad.
7. **Informationskvot:** genomsnittlig aktiv avkastning dividerad med tracking error, det vill säga Sharpe-kvoten beräknad på den avkastning som portföljen genererar utöver benchmark.
8. **Signal-till-brusförhållande för faktorpremier (SNR):** beräknas som $\frac{|\text{mean}(\lambda_{j,t})|}{\text{std}(\lambda_{j,t})}$ för faktor j . Ett SNR över 2 tyder på att premien är urskiljbar från brus.

9. **Informationskoefficient (IC):** mäter, för varje månad, Spearmans rangkorrelation mellan faktorvärden och realiserade framtida avkastningar över alla instrument i tvärsnittet.

$IC_{j,t} = 1 - \frac{\sum_i d_i^2}{n(n^2-1)}$, där d_i är skillnaden i rang mellan instrument i 's faktorvärde $F_{i,j,t}$ och dess framtida avkastning $R_{i,t+h}$ och n är antalet instrument i typsnittet vid tidpunkt t (Limor & Yakhini, 2025).

3.9 Sektormodell

I sektormodellen delades investeringsuniversumet in i tio sektorer i enlighet med tabell 3. Det som skiljde sektormodellen från Full-Universe är att faktorerna standardiserades tvärsnittsmässigt inom respektive sektor och månad. Tvärsnittsregressioner skattades för varje sektor separat och eftersom sambandet mellan faktorer och avkastningar varierade mellan sektorer erhöles sektorspecifika faktorpremier.

Baserat på de skattade avkastningarna byggdes en rangordning inom varje sektor. Endast de aktier med högst skattad avkastning inkluderades i portföljkonstruktionen enligt förvalda parametrar för antalet aktier i varje sektor. Därefter byggdes en separat portfölj per sektor genom en Markowitz-optimering enligt tidigare restriktioner, som beskrivs i 3.7. Riskmodellen är sektorbaserad och ges av (13), där B är faktorexponeringsmatrisen inom varje sektor och resterande termer är enligt tidigare.

Sist aggregerades de sektorvisa portföljerna till en total portfölj genom en sektorallokering. Vikterna som varje sektor tilldelades bestämdes av respektive sektors förväntade avkastning. Detta innebar att kapital allokerades proportionellt till de sektorer som uppvisade starkast signaler i den aktuella testperioden.

4 Resultat

I det här avsnittet presenteras resultaten. Den aggregerade modellen som erhöles efter sammanslagning av de individuella faktormodellerna var den mest kompletta samt bäst presterande modell som skapades under arbetet. Det framgår även att få faktorer hade en signifikant prediktionsförmåga och att de varierade kraftigt mellan sektorerna. Slutligen framgår att θ har ett optimalt värde och att EMA-spann påverkar resultatet marginellt.

4.1 Backtestperiod och inställningar

De två modellerna som beskrivs i 3.7.1 utvärderas nedan.

De resultat som presenteras nedan avser perioden 2019–2025 på ett universum av cirka 1000 aktier från nordamerikanska aktiemarknaden.

4.2 Faktoruppsättning

De åtta faktorer som användes för både Full-Universe- och sektormodellerna listas nedan:

1. Gross Margin (bruttomarginal)
2. ROIC (avkastning på investerat kapital)
3. P/FCF (pris per fritt kassaflöde)
4. EV/EBIT (företagsvärde per rörelseresultat)
5. Asset Growth (tillgångstillväxt)
6. Börsvärde (Log Market Cap, storleksfaktor)
7. OCF Margin (operativ kassaflödesmarginal)
8. Momentum (12-månaders prismomentum)

4.3 Portföljprestanda: Full-Universe vs. Sektor vs. Benchmark

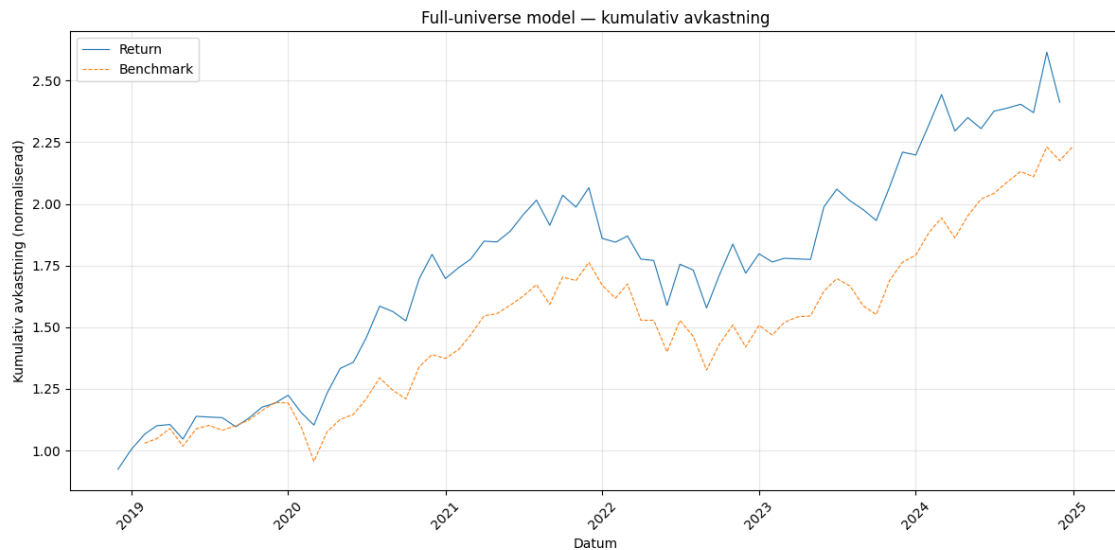
Portföljernas avkastning samt relevant presentationsmätning beräknades för Full-Universe och sektormodellen och jämfördes med S&P 500 samt en equal-weight-portfölj under perioden 2019–2025. Resultaten presenteras nedan.

Tabell 1: Prestanda för Full-Universe-modellen jämfört med S&P 500-benchmark för perioden 2019-2025.

Mått	Full-Universe	Sektor	Equal-Weight	S&P 500
Kumulativ avkastning	140.09 %	122.94 %	108.10 %	~ 137 %
Annualiserad avkastning	15.95 %	14.09 %	12.08 %	~ 15.5 %
Annualiserad volatilitet	18.34 %	23.27 %	18.52 %	~ 18 %
Sharpe-kvot	0.87	0.68	0.75	~ 0.75
Max Drawdown	−23.61%	−19.71 %	−18.9 %	~ −25-30 %
Information Ratio	0.21	0.31	0.40	-
Tracking Error	~8.66 %	16.23 %	7.12 %	-
Antal observationsperioder	71	73	73	-

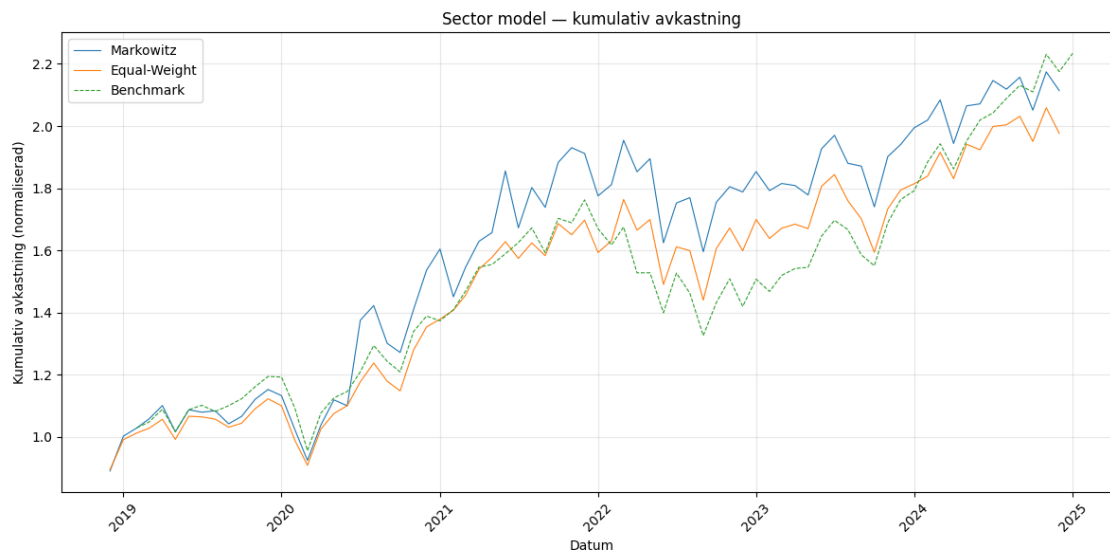
^a Full-Universe-modellen har 71 observationsperioder jämfört med 73 för övriga portföljer. Skillnaden beror på att modellen kräver ett längre initialiseringsförfarande vid uppstart av optimeringen.

Figur 1 illustrerar portföljens relativa avkastningsutveckling under backtestperioden för Full-Universe-modellen. Det framgår att modellen är följsam till index speciellt under 2019–2020, och att den därefter i stort reflekterar dess prISRörelse med viss överavkastning.



Figur 1: Full-Universe avkastning gentemot S&P 500-benchmark. Y-axeln visar den kumulativa avkastningen indexerad till 1.0 vid startdatumet, och x-axeln visar tid i år.

Figur 2 visar sektormodellens utveckling under samma period, med S&P 500 och Equal-Weight som referensportföljer. Portföljerna har principiellt identisk värdeutveckling under de första arton månaderna, varefter sektormodellen konsekvent slår övriga portföljer fram tills de allra sista månaderna då index överskrider Markowitzportföljen.



Figur 2: Jämförelse mellan sektormodell, Equal-Weight-portfölj och S&P 500-benchmark. På y-axeln visas den kumulativa avkastningen och x-axeln visar tid i form av år. Den kumulativa avkastningen är normaliserad med startvärde 1.0.

Full-Universe-modellen uppnår en kumulativ avkastning på ungefär 140% över backtestperioden och sektormodellen omkring 123% under samma period. Båda överträffar Equal-Weight portföljen med 15-32 procentenheter, men endast Full-universe uppnår överavkastning relativt S&P500-indexet, om än med liten marginal. Sektormodellen tar märkbart högre risk än övriga portföljer, vilket återspeglas i Sharpe-kvoten, hög annualiserad volatilitet och stor Tracking Error.

4.4 Faktorerers prediktionsförmåga och Sharpe-kvotens beroende på faktoruppsättning

Följande tabeller visar antalet 5 %-signifikanta faktorer per sektor samt de mest prediktiva faktorerna i respektive sektor. Sektorvikter finns presenterade i Appendix B.1.

Tabell 2: Faktorernas statistiska signifikans och Signal-To-Noise ratio i **Full-Universe**-modellen.

Faktor	Genomsnittligt t-stat	SNR	Status
Asset Growth	2.70	0.277	Signifikant
ROIC	2.15	0.215	Signifikant
P/FCF	1.89	0.189	Marginellt signifikant
EV/EBIT	1.67	0.167	Marginellt signifikant
Börsvärde	0.52	<0.1	Insignifikant
OCF Margin	-0.46	<0.1	Insignifikant
Gross Margin	-0.67	<0.1	Insignifikant
Momentum	0.90	<0.1	Insignifikant

Tabell 2 visar att Asset Growth, ROIC och P/FCF är signifikanta och ger starkast signaler när de aggregeras över hela investeringsuniversumet. Det är värt att notera att endast två av åtta faktorer uppnår statistisk signifikans på 5%-nivån, vilket indikerar att majoriteten av de valda faktorerna inte bär tillräcklig prediktiv information när de estimeras över hela investeringsuniversumet. Detta står i kontrast till tidigare studier på den amerikanska marknaden där ROIC och Asset Growth bland andra konsekvent visat starkare signaler (Novy-Marx, 2013; Hou m. fl., 2015). En liknande analys för varje sektor följer nedan:

Tabell 3: Antal signifikanta faktorer och starkaste prediktorer per sektor i **sektormodellen**.

Sektor	Sign. faktorer	Starkaste faktor	T-stat
Dagligvaror	1/8	Gross Margin	-2.39
Energi	0/8	P/FCF	1.8
Finans & Fastighet	0/8	EV/EBIT	-1.64
Hälsovård	3/8	ROIC	3.72
Industri	2/8	OCF Margin	3.16
Informationsteknik	0/8	ROIC	1.55
Kraftförsörjning	0/8	Börsvärde	-1.77
Material	1/8	ROIC	-3.62
Sällanköpsvaror	3/8	ROIC	3.45
Telekommunikation	1/8	P/FCF	-2.68

Ur tabellen framgår att ROIC är en robust faktor i många sektorer (hälsovård, informationsteknik och sällanköpsvaror) men uppvisar motsatt riktning (negativt t-stat) i Material. Slutligen illustrerar nedanstående tabell hur slumpade uppsättningar av ett ökande antal faktorer påverkat den riskjusterade avkastningen för Full-universe-modellen:

Antal faktorer	Sharp-kvoten	Faktorer i kombinationen
1	0.20	ROIC
1	N/A	GrossMargin
1	N/A	Börsvärde
2	0.68	ROIC, EV/EBIT
2	0.89	ROIC, P/FCF
2	0.61	GrossMargin, Börsvärde
3	0.31	EV/EBIT, AssetGrowth, OCF Margin
3	0.20	GrossMargin, P/FCF, EV/EBIT
3	0.39	P/FCF, Börsvärde, OCF Margin
4	0.57	AssetGrowth, Börsvärde, OCF Margin, Momentum
4	0.92	GrossMargin, ROIC, AssetGrowth, Momentum
4	0.76	ROIC, Börsvärde, OCF Margin, Momentum
5	0.36	GrossMargin, ROIC, P/FCF, EV/EBIT, OCF Margin
5	0.08	GrossMargin, ROIC, P/FCF, EV/EBIT, Börsvärde
5	0.45	GrossMargin, ROIC, P/FCF, AssetGrowth, OCF Margin
6	0.45	GrossMargin, ROIC, P/FCF, AssetGrowth, Börsvärde, OCF Margin
6	0.90	GrossMargin, ROIC, P/FCF, AssetGrowth, Börsvärde, Momentum
6	0.77	GrossMargin, P/FCF, EV/EBIT, AssetGrowth, Börsvärde, Momentum
7	0.55	GrossMargin, ROIC, P/FCF, EV/EBIT, AssetGrowth, Börsvärde, OCF Margin
7	0.79	GrossMargin, ROIC, P/FCF, AssetGrowth, Börsvärde, OCF Margin, Momentum
7	0.90	GrossMargin, ROIC, P/FCF, EV/EBIT, AssetGrowth, Börsvärde, Momentum
8	0.94	GrossMargin, ROIC, P/FCF, EV/EBIT, AssetGrowth, Börsvärde, OCF Margin, Momentum

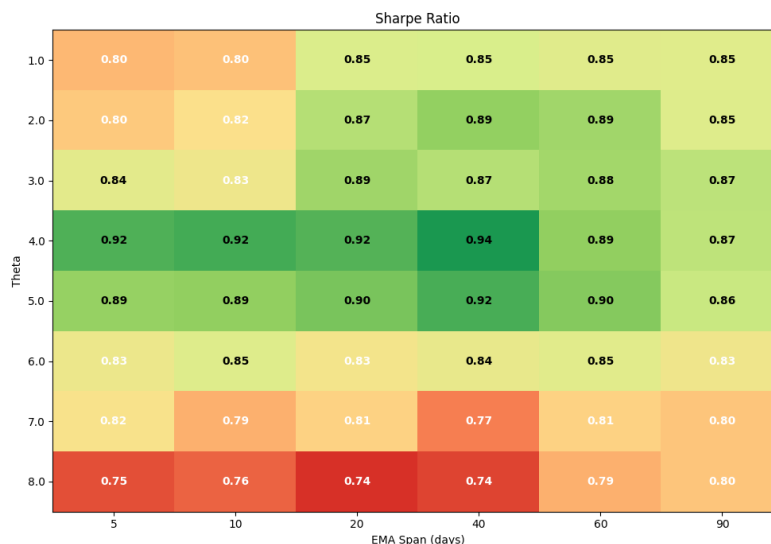
Tabell 4: Sharpe-kvoter för olika faktorkombinationer

Tabell 4 tycks indikera att ett större antal faktorer i modellen resulterar i en högre riskjusterad avkastning. Värt att poängtera är dock att även lägre antal faktorer i kombination ibland ger högre Sharpe-kvot än körningar som använder fler faktorer. Exempelvis framgår att ROIC +

P/FCF i kombination effektivt dubblat den riskjusterade avkastningen relativt körningen med GrossMargin, ROIC, P/FCF, AssetGrowth och OCF Margin i kombination (Sharpe-kvot 0.89 mot 0.45).

4.5 Riskaversionens påverkan

För att undersöka hur varierande grader av risktagande påverkade modellprestationer simulerades **Full-universe**-modellen med olika θ -värden och EMA-spänn. För varje kombination av θ och EMA-spänn noterades Sharpe-kvoten som prestationsmått. Resultaten presenteras i nedanstående heatmap:



Figur 3: Full-Universe parameteranalys av θ och EMA-spännets effekt på Sharpe-kvot

Sharpe-kvoten maximeras för $\theta \in (4, 5)$ medan $\theta > 6$ sänker kvoten markant; det senare indikerar alltså en överdrivet konservativ investeringsstrategi. Heatmapen visar även att EMA-spänn ≥ 40 verkar ge marginellt bättre resultat. En längre minnestid minskar kortsiktigt brus i faktorerna, vilket kan ge stabilare signaler men samtidigt introducera en viss tröghet i modellen, som gör den mindre kapabel till att reagera på plötsliga, kraftiga trendsiften i marknaden.

5 Diskussion

I detta avsnitt diskuteras studiens frågeställningar i relation till resultaten. Mer specifikt utvärderas modellernas prestation samt hur denna påverkas av faktorval, hyperparametrar och risktagande. Avslutningsvis behandlas studiens begränsningar och hur alternativa metodval hade kunnat påverka resultatet.

5.1 Besvarar modellen studiens frågeställningar?

Nedan diskuteras huruvida rapportens frågeställningar kan besvaras utifrån studiens metodiska undersökning och producerade resultat.

5.1.1 Fråga 1: Vilken annualiserad avkastning kan uppnås?

Full-Universe-modellen uppnår en annualiserad avkastning på 15.95 % och sektormodellen 14.09%, jämfört med S&P-indexets 15.5 %. Över backtestperioden innebär detta en överavkastning på ungefär tre procentenheter för Full-Universe-modellen och en underavkastning på ungefär fjorton procentenheter för sektormodellen, relativt benchmark.

Full-Universe-modellen uppvisar högre riskjusterad avkastning än benchmark (Sharpe-kvot 0.87 mot 0.75), vilket tycks indikera att överavkastningen inte enbart är ett resultat av högre risktagande. De annualiserade volatiliteterna är i paritet med varandra och detsamma kan sägas för båda portföljernas maximum drawdown. Tabell 1 illustrerar vidare att Markowitz portföljoptimering genererar ett mervärde: för varje enhet av risk som tas i ett Markowitz-ramverk, genereras mer avkastning relativt samma riskbenägenhet i en Equal-Weight-portfölj. Detta stöds av Full-Universe:s överavkastning relativt Equal-Weight-portföljen vid i princip identiska annualiserade volatiliteter. Det bör dock noteras att Sharpe-kvoterna bygger på skattningar av avkastning och volatilitet, vilket innebär att den marginella prestandaskillnaden mellan Full-Universe och S&P 500 lika gärna kan förklaras av statistiskt brus som av en systematisk överprestation. Om prestationsmåttene hade skiljt sig avsevärt mellan modellen och benchmarkindexet hade modellen i större utsträckning kunnat tillskrivas förmågan att generera högre riskjusterad avkastning. Eftersom skillnaden däremot är begränsad kan någon sådan slutsats inte dras med säkerhet, då den observerade överprestationen kan vara ett resultat av stokastisk variation i data snarare än genuin prediktionsförmåga.

Sektormodellen tar högst risk (23.27 %) och har en tracking error som effektivt dubblar övriga portföljers (16.23 %). Markowitz ger oss fortfarande ett mervärde relativt en likafördelad portfölj, men här på bekostnad av högre annualiserad volatilitet och större avvikelse från benchmark. Den större riskbenägenheten resulterar i en relativt liten överavkastning jämfört med Equal-Weight-portföljen, vilket återspeglas i dynamiken mellan dessas Sharpe-kvoter: 0.68 mot 0.75. Sektormodellen kan därvid inte rättfärdiga sitt risktagande utifrån riskjusterad avkastning. Sektormodellen presterar alltså sämre än en Equal-Weight portfölj när risk beaktas, över denna backtestperiod. Nämnvärt här är dock det som figur 2 illustrerar: sektormodellens till synes svaga prestanda gentemot referensportföljer härstammar från backtestperiodens sista skede. Det framgår av grafen att under merparten av backtesting-perioden har sektormodellen högre avkastning än Equal-Weight och benchmark. En annan backtesting-period hade sannolikt framställt sektormodellen som starkare än vad den framstår över perioden 2019–2025.

En trolig förklaring till sektormodellens underprestation relativt Full-Universe-modellen är att sektorvis Markowitz-optimering introducerar ett dubbelt skattningsproblem: dels måste förväntade avkastningar skattades för varje sektor, dels måste kapital allokeras mellan sektorer baserat på dessa estimat. Varje sådant steg medför skattningsfel som ackumuleras i den slutliga portföljkonstruktionen. I Full-Universe-modellen optimeras alla tillgångar i ett och samma steg, vilket reducerar antalet frihetsgrader och därmed den totala skattningsosäkerheten. Därtill visar tabell 3 att många sektorer saknar statistiskt signifikanta faktorer överhuvudtaget — i sådana sektorer genererar regressionen huvudsakligen brus, och Markowitz-optimeraren allokerar kapital baserat på vad som i praktiken är slumpmässiga signaler. Detta är sannolikt den primära drivkraften bakom sektormodellens högre volatilitet och lägre Sharpe-kvot.

Ovanstående resultat ligger i linje med vad som observerats i liknande empiriska studier. Grinold och Kahn påpekar att sektorvis portföljoptimering typiskt kräver robustare faktorsignaler inom varje sektor för att generera stabil överavkastning (Grinold och Kahn, 2000) — en förutsättning som vår modell uppenbarligen inte till fullo uppfyller givet de låga t-statistikor som återfinns i exempelvis Energi och Kraftförsörjning. På liknande sätt har det visats att faktorbaserade strategier med svaga eller instabila signaler tenderar att prestera sämre än enkla marknadskapitalviktade index, i synnerhet när transaktionskostnader och ombalanseringsfrekvens beaktas (Arnott m. fl., 2016).

5.1.2 Fråga 2: Vilka faktorer har bäst prediktiv förmåga och hur påverkar faktoruppsättning Sharp-kvoten?

Resultaten i tabell 1 visar att den aggregerade modellen, under användning av alla åtta valda faktorer, genererade bäst riskjusterad avkastning (Sharpe-kvot 0.87). Det ter sig därmed fördelaktigt att inkludera fler faktorer, snarare än färre, i en regressionsmodell. Vad beträffar specifika faktorkombinationer och deras effekt på Sharpe-kvoten, så har vi sett att Sharpe-kvoten ökar med antalet faktorer i regressionsmodellen (se tabell 4). Det bör dock noteras som en metodologisk begränsning

att systematiska tester av alla möjliga faktorkombinationer inte genomfördes. Slutsatsen att fler faktorer ger bättre resultat baseras på sekventiell faktoraddition, inte en komplett kombinatorisk undersökning. Det är därför möjligt att en delmängd av de åtta faktorerna hade givit likvärdig eller bättre Sharpe-kvot med lägre modellkomplexitet.

Som nämnt i avsnitt 4.2 valdes åtta faktorer i syfte att förklara företags **value, growth** och **performance**. Fler faktorer ger en mer heltäckande bild av bolagens finansiella karakteristik; åtminstone upp till och med åtta faktorer. En naturlig utvidgning är att undersöka hur fortsatt faktoraddition påverkar Sharpe-kvoten för regressionsmodellen. Akademisk litteratur (Fama och French, 2015, Arnott m. fl., 2016) pekar på att kombinerandet av många svagt signalerande faktorer kan introducera en *dimensionality-curse*: att brus ackumuleras snabbare än informationsvärde, vilket leder till högre transaktionskostnader utan motsvarande avkastningsförbättring. Om en övre gräns för antalet nyttiga faktorer existerar har vi uppenbarligen inte nått den inom omfånget av denna studie. Sektormodellens signaldiagnostik tycks styrka hypotesen om avtagande marginalnytta: antalet signifikanta faktorer överstiger aldrig tre av åtta per sektor (tabell 3), och det aggregerade SNR-förhållandet är lågt för majoriteten av faktorerna.

Det förhållandevis begränsade faktorurvalet gjorde det svårt att urskilja vilka specifika kombinationer som hade bäst förklaringskraft, både aggregerat och inom enskilda sektorer. En utvidgning av faktorurvalet och fortsatt undersökning av sektorspecifik förklaringskraft hade gett tydligare svar på frågeställning 2. Trots denna oklarhet visar tabellerna 1 och 3 att enskilda faktorer har god prediktionsförmåga. ROIC och P/FCF är starka i flertalet sektorer samt signifikanta över hela investeringsuniversumet. Asset Growth är i särklass den starkaste faktorn sett till signalstyrka och SNR utanför sektorkörningen.

Att Asset Growth är en stark faktor överensstämmer med väletablerad finansiell teori. Fama och French argumenterar för att högtillväxtbolag typiskt investerar tungt på hög kapitalbekostnad, och att marknaden systematiskt överskattar avkastningen på dessa investeringar medan den undskattar avkastningen för mindre tillväxtfokuserade bolag (Fama och French, 2015). Denna investeringsanomali är en av de mest robusta faktorpremierna på moderna finansmarknader och har validerats över flera marknader och tidsperioder (Hou m. fl., 2015; Arnott m. fl., 2016). Den praktiska innebörden av detta är att sakta växande bolag konsekvent slår snabbväxande bolag i termer av avkastning — en motsats till intuitionen men en väldokumenterad empirisk realitet.

Gross Margin och OCF Margin är bland de faktorer med svagast prediktionsstyrka, vilket inte är förvånande i ljuset av befintlig forskning. Asset Growth och ROIC fångar aspekter av företags kapitalallokering och operativ effektivitet som dokumenterats ha robust prediktiv kraft över långa tidsperioder och på flera marknader (Fama och French, 2015; Novy-Marx, 2013). Marginalmåtten Gross Margin och OCF Margin är däremot mer känsliga för redovisningsskillnader mellan sektorer. De tenderar att korrelera med varandra och med andra faktorer i modellen, vilket kan leda till kolinearitet och dämpade eller instabila koefficienter i regressionen. En alternativ förklaring är att dessa marginalmått redan till stor del är inprisade i marknaden och saknar den typen av informationsförspång som ger upphov till systematisk överavkastning. Det negativa tecknet på t-statistikan för dessa faktorer bör inte tolkas som att höga brutto- och kassaflödesmarginaler straffas av marknaden. Det är snarare ett symptom på instabila koefficienter till följd av kolineariteten.

Utöver brutto- och kassaflödesmarginalerna är resterande faktorer också svaga i flera sektorer. En möjlig förklaring är att vår testperiod (2019–2025) inkluderar perioder med extrema marknadsförhållanden, inklusive Covid-19-pandemin, vilket kan ha ökat faktorernas tidsvariation och därmed standardavvikelsen i Fama-MacBeth-skattningarna. Ett särskilt fall värt att lyfta fram är informationsteknik, som uppvisar 0/8 signifikanta faktorer trots att sektorn historiskt har genererat betydande överavkastning relativt övriga marknaden. En trolig förklaring är att IT-sektorn karaktäriseras av snabba och abrupta strukturskiften, drivna av teknologisk exponentiell tillväxt och stora informationsflöden. Kursutvecklingen för bolag som NVIDIA och Apple påverkas kraftigt av nyheter på kort sikt och sektorns värdering bygger i hög grad på förväntade framtida kassaflöden snarare än nuvarande fundamenta. Här fungerar exempelvis prismomentum bättre än vad

det gör i klassiska industrisektorer som traditionellt uppvisar mindre trendberoende. Priset drivs därvid i stor utsträckning av marknadens sentiment och förväntningar, snarare än av de balansräkningsbaserade faktorerna som modellen bygger på. Detta illustrerar en principiell begränsning hos faktormodeller i allmänhet: de är bättre anpassade till sektorer där pris i högre grad reflekterar ett bolags traditionella finanser, och sämre lämpade för sektorer med hög nyhetsdriven volatilitet, där värderingar baseras på framtida kassaflöden.

En naturlig vidareutveckling av sektormodellen hade varit att konstruera sektorspecifika faktoruppsättningar, där enbart de faktorer med statistiskt signifikant prediktiv förmåga inom respektive sektor inkluderas, i enlighet med Tabell 3. Exempelvis domineras hälsovård och sällanköpsvaror av ROIC medan industri drivs av OCF Margin — mönster som går förlorade när samma åtta faktorer tillämpas uniformt över alla sektorer. Baserat på denna studies faktorurval, hade det därmed varit motiverat att helt exkludera sektorer där ingen faktor uppnår statistisk signifikans, såsom Energi och Kraftförsörjning, eftersom Markowitz-optimeraren i dessa fall allokerar kapital baserat på vad som i praktiken är brus snarare än verkliga signaler. Dessa åtgärder hade sannolikt reducerat den höga volatiliteten och det dubbla skattningsproblemet som identifierats som primära drivkrafter bakom sektormodellens underprestation.

En ytterligare utvidgning av sektormodellen är att använda en annan regressionsmetod. Fama-MacBeth skattar faktorpremier som tidsmedelvärden av tvärsnittsregressioner, vilket gör metoden känslig för tidsvariation i faktorpremierna. En poolad panel-OLS hade istället kunnat fånga sådana tidsvarierande effekter mer explicit och hade potentiellt givit stabilare skattningar av faktorpremier i sektorer där få observationer finns tillgängliga per tidpunkt. Det hade kunnat förbättra de låga t -statistikorna i exempelvis Energi och Kraftförsörjning. Framtida arbete bör därför undersöka huruvida en övergång till poolad panel-OLS förändrar faktorrankningen eller förbättrar modellens prediktiva förmåga.

5.1.3 Fråga 3: Hur påverkar hyperparametrar modellens prestanda?

Att Sharpe-kvoten maximeras för $\theta \in (4, 5)$ är ett intuitivt rimligt resultat. Alltför lågt θ innebär att optimeraren tar oproportionerligt hög risk givet de faktorsignaler som motiverar förväntad avkastning. Alltför högt θ leder omvänt till att portföljen konvergerar mot en nästintill likafördelad allokering med låg aktiv risk och avkastning. Det optimala θ -intervallet kan tolkas som den riskaversion vid vilken faktorsignalernas informationsinnehåll bäst omsätts i riskjusterad avkastning.

Vad gäller EMA-spannet indikerar resultaten att ett spann på minst 40 dagar är fördelaktigt. Den praktiska implikationen är att modellen bör tillåtas integrera fundamentala nyckeltal över en något längre tidshorisont, vilket ger mindre vikt till kvartalsvisa fluktuationer och behåller fokus på bolagens fundamentala karakteristik. Trots att resultaten talar för ett längre EMA-spann, bör det poängteras att handel inom sektorer med stora informationsflöden kan stå att gagnas av motsatsen. Som bekant gör en längre minnestid att modellen jämnar ut kortsiktiga volatila svängningar, vilket kan påverka faktorsignifikans i nyhetsdrivna sektorer. Vidare studier kan därmed undersöka hur längden på EMA-spannet påverkar faktorers signifikans i sektorer med hög nyhetsdriven volatilitet.

Det existerar slutligen ett ekonomiskt incitament bakom konstruktionen av faktormodeller, varför en robust och reproducerbar modell eftersträvas. Prestationen ska inte märkbart försämrats vid små variationer av hyperparametrar, som exempelvis EMA-spannet. Ett sådant beroende tyder på en instabilitet och begränsad generaliserbarhet: det finns inga garantier för att modellen presterar som väntat under en annan tidshorisont. Ur figur 3 framgår det att om EMA-spannet varieras påverkas inte Sharpe-kvoten nämnvärt, vilket tyder på en viss robusthet.

5.2 Slutsats

Detta arbete har undersökt huruvida faktorbaserad portföljoptimering kan generera riskjusterad avkastning genom att kombinera Fama-MacBeth-regression som metod för faktorestimering och Markowitz som optimeringsteknik.

Resultat visar att Full-Universe-modellen uppnår en annualiserad avkastning på 15.95 % och en Sharpe-kvot på 0.87, jämfört med S&P 500:s 15.5 % och Sharpe-kvot på 0.75. Även om den optimerade portföljen uppvisar en högre skattad Sharpe-kvot än S&P 500, är skillnaden ytterst liten. Det kan inte med säkerhet konstateras att modellen genererar högre riskjusterad avkastning än benchmark enbart på grund utav god prediktionsförmåga, utan stokastiskt brus kan lika gärna ha gett upphov till marginalnyttan. Sektormodellen presterar något sämre än S&P 500, vilket sannolikt förklaras av ett begränsat antal signifikanta faktorer för de enskilda sektorerna.

Faktoranalysen identifierar Asset Growth, ROIC, P/FCF som mest robusta, medan OCF Margin och Gross Margin uppvisar lägre signifikans och bidrar i högre grad med brus i modellen. Hyperparameteranalysen visar att ett θ -värde mellan 4 och 5 samt ett EMA-spann på minst 40 dagar ger de mest stabila faktorsignalerna och den högsta Sharpe-kvoten.

Sammanfattningsvis visar resultaten att en faktorbaserad portföljstrategi i kombination med Markowitz optimeringsramverk kan generera konkurrenskraftig riskjusterad avkastning. Samtidigt indikerar resultaten att marginalerna mot marknadsindex är små och känsliga för val av faktorer och modellparametrar samt att signifikansen hos ett nyckeltal kan ha stor påverkan på resultatet. Detta understryker vikten av statistisk osäkerhet vid utvärdering av överavkastning. Framtida arbete kan involvera sektorspecifika nyckeltal samt alternativa regressionsmetoder för att förbättra signalstyrkan och öka stabiliteten i överavkastningen.

Referenser

- Anthropic. (2026). Claude [Accessed: 2026-01-15 to 2026-05-11].
- Arnott, R. D., Beck, N., Kalesnik, V., & West, J. (2016). How Can “Smart Beta” Go Horribly Wrong? *The Journal of Portfolio Management*, 42(2), 31–43. <https://doi.org/10.3905/jpm.2016.42.2.031>
- Fama, E. F., & French, K. R. (1992). The Cross-Section of Expected Stock Returns. *The Journal of Finance*, 47(2), 427–465. <https://doi.org/10.1111/j.1540-6261.1992.tb04398.x>
- Fama, E. F., & French, K. R. (1993). Common Risk Factors in the Returns on Stocks and Bonds. *Journal of Financial Economics*, 33(1), 3–56. [https://doi.org/10.1016/0304-405X\(93\)90023-5](https://doi.org/10.1016/0304-405X(93)90023-5)
- Fama, E. F., & French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics*, 116(1), 1–22. <https://doi.org/10.1016/j.jfineco.2014.10.010>
- Fama, E. F., & Macbeth, J. D. (1973). Risk, Return, and Equilibrium: Empirical Tests. *The Journal of Political Economy*, 83(3), 607–636.
- Graham, B., & Dodd, D. L. (1934). *Security Analysis*. McGraw-Hill.
- Grinold, R. C., & Kahn, R. N. (2000). *Active Portfolio Management: A Quantitative Approach for Providing Superior Returns and Controlling Risk* (2. utg.). McGraw-Hill.
- Guo, K., Wang, W., Tian, S., Yang, J., Jiang, Z., & Dai, Z. (2022). Research on Optimization Technology of Cross-Regional Synergistic Deployment of Fire Stations Based on Fire Risk. *Sustainability*, 14(23), 15725. <https://doi.org/10.3390/su142315725>
- Hou, K., Xue, C., & Zhang, L. (2015). Digesting Anomalies: An Investment Approach. *The Review of Financial Studies*, 28(3), 650–705. <https://doi.org/10.1093/rfs/hhu068>
- Ledoit, O., & Wolf, M. (2004). A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariabel Analysis*, 88(2), 365–411. [https://doi.org/10.1016/S0047-259X\(03\)00096-4](https://doi.org/10.1016/S0047-259X(03)00096-4)
- Lien, D., & Balakrishnan, N. (2023). Some results on multiple regression analysis with data cleaned by trimming and winsorization. *Communications in Statistics - Simulation and Computation*, 52(10), 5082–5089. <https://doi.org/10.1080/03610918.2021.1982974>
- Limor, O., & Yakhini, Z. (2025). An Interpretation of Spearman Correlation via k-Subset Permutations. *2025 IEE International Conference on High Performance Computing and Communications (HPCC)*, *High Performance Computing and Communications (HPCC)*, 1–9. <https://doi.org/10.1109/HPCC67675.2025.00146>
- Markowitz, H. M. (1952). Portfolio selection. *Journal of Finance*, 7(1), 77–91. <https://doi.org/10.14400/JDC.2022.20.4.185>
- Milton, J. S., & Arnold, J. C. (2003). *Introduction to Probability and Statistics: Principles and Applications for Engineering and the Computing Sciences* (4. utg.). McGraw-Hill.
- Morales-Brotons, D., Vogels, T., & Hendriks, H. (2024). Exponential Moving Average of Weights in Deep Learning: Dynamics and Benefits. *Transactions on Machine Learning Research*. <https://doi.org/10.48550/arXiv.2411.18704>
- Novy-Marx, R. (2013). The other side of value: The gross profitability premium. *Journal of Financial Economics*, 108(1), 1–28. <https://doi.org/10.1016/j.jfineco.2013.01.003>
- OpenAI. (2026). ChatGPT [Accessed: 2026-01-15 to 2026-05-11].
- West, R. M. (2022). Best practice in statistics: The use of log transformation. *Annals of Clinical Biochemistry*, 59(3), 162–165. <https://doi.org/10.1177/00045632211050531>

AI-användande

Under arbetets gång har AI (Claude (Anthropic, 2026), ChatGPT (OpenAI, 2026)) använts på flera sätt. Bland annat för att diskutera idéer och fördjupa förståelsen kring faktormodeller, vilka faktorer som är vanligast och olika typer av regressionsmetoder. Under kodningen har AI använts för att tolka felmeddelanden som framkommit och verifiera att ingen look-ahead-bias introducerats. Under skrivprocessen har AI använts för hjälp med textformatering: att skriva matematiska formler, formatera tabeller och se över dokumentets struktur. Allt detta för att spara tid, stödja utvecklingen av nya idéer, jobba mer med modern teknik och säkerställa resultaten.

A Kompletterande Teori

A.1 Z-score

Z-score används för att skala om data och göra den jämförbar över olika storleksordningar. Z-score ges på formen:

$$Z = \frac{x - \mu}{\sigma} \quad (14)$$

Där x är datapunkten, μ är medelvärdet och σ är standardavvikelsen. (Milton & Arnold, 2003). Måttet kan tolkas enligt följande: $Z = 0$ tolkas som att x ligger exakt på medelvärdet av dess dataserie, och $Z = \pm 1$ motsvarar värden som ligger en standardavvikelse under respektive över medelvärdet.

A.2 Rang-standardisering

Rang-standardisering bearbetar data genom att skala om alla datapunkter och ge dem en rang i intervallet $[-1, 1]$. För att utföra Rang-standardisering på en faktor F_j för instrument i vid tidpunkt t används följande formel (Guo m. fl., 2022):

$$\tilde{F}_{i,j,t} = \frac{F_{i,j,t} - \min_i(F_{j,t})}{\max_i(F_{j,t}) - \min_i(F_{j,t})}$$

där $\min_i(F_{j,t})$ och $\max_i(F_{j,t})$ betecknar det minsta respektive största observerade faktorvärdet i tvärsnittet av alla instrument vid tidpunkt t . Nu finns alla rangordningar på intervallet $[0, 1]$ och för att skala om till intervallet $[-1, 1]$, görs varje rangordning om enligt:

$$\hat{F}_{i,j,t} = 2\tilde{F}_{i,j,t} - 1.$$

A.3 Exponentiell viktning (EMA)

Det exponentiella glidande medelvärdet används för att jämna ut kortsiktiga fluktuationer i de rang-standardiserade faktorvärdena. Metoden ger ny data exponentiellt högre vikt än äldre observationer, vilket gör att modellen anpassar sig gradvis till ny information. Det exponentiellt viktade medelvärdet definieras rekursivt för varje instrument enligt (Morales-Brotons m. fl., 2024):

$$EMA_t = \alpha \cdot F_{i,t} + (1 - \alpha) \cdot EMA_{t-1}, \quad \alpha = \frac{2}{\text{span} + 1}$$

där span är en parameter för minneslängden. Ett högt span innebär att äldre observationer ges relativt stor vikt, medan ett mindre span gör modellen mer reaktiv men också mer känslig för brus.

A.4 Kostnad för handel

Vid varje ombalansering av portföljen uppkommer det transaktionskostnader i form av spread (skillnaden mellan köp- och säljpris), slippage (prisglidning vid stora ordervolymer) och avgifter. Transaktionskostnader inkluderas som en straffterm i målfunktionen för att förhindra hög turnover:

$$\max_{\pi} \mathbb{E}[R^{\pi}] - \theta \cdot \text{Var}(R^{\pi}) - \gamma \cdot \sum_i |\pi_{i,t} - \pi_{i,t-1}|, \quad (15)$$

där $\gamma = 0.05 \cdot \sigma$ och $\sum_i |\pi_{i,t} - \pi_{i,t-1}|$ är portföljens turnover. Transaktionskostnader blir effektivt en straffmekanism för risk, då det medför högre handelsavgifter för volatila aktier, då dessa handlas mer frekvent än mindre volatila tillgångar.

B Utökade resultat

B.1 Sektorvikter

I nedanstående tabell framgår hur stor andel av det totala kapitalet som sektormodellen allokerade till respektive marknadssektor.

Tabell 5: Portföljinnehav utefter marknadssektor i sektormodellen.

Sektor	Genomsnittlig vikt
Hälsovård	13.20 %
Kraftförsörjning	12.05 %
Informationsteknik	11.77 %
Dagligvaror	11.30 %
Finans & Fastighet	11.17 %
Sällanköpsvaror	11.04 %
Industri	10.89 %
Material	9.57 %
Energi	9.01 %
Total	100.00 %

Ur tabellen framgår att trots varierande grad av signifikanta faktorer - sektorer emellan - är kapitalallokeringen tämligen jämn över samtliga sektorer.

C Matematiska bevis

Nedan bifogas ett bevisande resonemang som redogör för hur den optimala portföljstrategin i Markowitz optimeringsproblem ska konstrueras.

C.1 Bevis optimal portfölj

Notera att för en godtycklig strategi π gäller

$$g(\pi) = g(\pi^*) + \sum_i \frac{\partial g}{\partial \pi_i}(\pi^*)(\pi_i - \pi_i^*) - \theta \sum_{i,j} (\pi_i - \pi_i^*)(\pi_j - \pi_j^*) c_{i,j}$$

och eftersom $\partial_{\pi_i} g(\pi^*) = 0 \quad \forall i$ per konstruktion, samt att det senare ledet ovan kan skrivas som $\theta \cdot \text{Var}(\sum_i (\pi_i - \pi_i^*) R_i) \geq 0$, följer det att $g(\pi) \leq g(\pi^*)$ för samtliga π . Alltså är $g(\pi^*)$ ett globalt maximum för målfunktionen (5).

D Källkod

Nedan följer den källkod skriven i python som används för att hämta data samt för att träna och utvärdera modellerna. Notera att en API nyckel på börsdata krävs för att använda funktionerna som hämtar den finansiella datan för de givna instrumenten.

```
1 class Faktormetoder:
2
3     def __init__(self, api_key: str, kpi_dict: dict, data_dir: str) -> None:
4         self.api_key = api_key
5         self.kpi_dict = kpi_dict
6         self.data_dir = data_dir
7
8     def chunk_50(self, instrument_IDs: list, size: int = 50):
9
10        for i in range(0, len(instrument_IDs), size):
11            yield instrument_IDs[i:i + size]
12
13    def fetch_kpi_history(
14        self,
15        target_ids: list[int],
16        kpi_ids: list[int],
17        report_type: str = "r12",
18        count: int = 5
19    ) -> pd.DataFrame:
20
21
22        with BorsdataClient(self.api_key) as client:
23            instruments = client.get_instruments()
24            instrument_lookup = {i.ins_id: i.name for i in instruments}
25            rows = []
26
27            for kpi_id in kpi_ids:
28                for ids_batch in self.chunk_50(target_ids):
29                    resp = client.get_kpi_history_batch(
30                        instrument_ids=ids_batch,
31                        kpi_id=kpi_id,
32                        report_type=report_type,
33                        max_count=count
34                    )
35
36                    for kpi_hist in resp.kpis_list:
37                        ins_id = kpi_hist.instrument
38                        name = instrument_lookup.get(ins_id, f"Unknown ({ins_id})")
39
40                        for val in kpi_hist.values:
41                            period = val.y if report_type == "year" else f"{val.y}Q{val.p}"
42
43
44                            if report_type == "year":
45                                date = pd.to_datetime(f"{val.y}-12-31")
46                            else:
47                                date = (
48                                    pd.Period(f"{val.y}Q{val.p}", freq="Q")
49                                    .end_time.normalize().tz_localize(None)
50                                )
51
52                            rows.append({
53                                "Instrument": ins_id,
54                                "Name/Ticker": name,
55                                "Date": date,
```

```

55         "Period": period,
56         "KPI_ID": kpi_id,
57         "KPI": val.v
58     })
59
60     return pd.DataFrame(rows)
61
62     def priser(
63         self,
64         target_ids: list[int],
65         from_date: datetime,
66         to_date: datetime
67     ) -> pd.DataFrame:
68
69         with BorsdataClient(self.api_key) as client:
70             instruments = client.get_global_instruments()
71             instrument_lookup = {i.ins_id: i.name for i in instruments}
72             rows = []
73
74             for ids_batch in self.chunk_50(target_ids):
75                 prices_array = client.get_stock_prices_batch(
76                     instrument_ids=ids_batch,
77                     from_date=from_date,
78                     to_date=to_date
79                 )
80
81                 for inst_prices in prices_array:
82                     ins_id = inst_prices.instrument
83                     name = instrument_lookup.get(ins_id, f"Unknown ({ins_id})")
84
85                     if inst_prices.stockPricesList:
86                         for price in inst_prices.stockPricesList:
87                             rows.append({
88                                 "Date": datetime.strptime(price.d, "%Y-%m-%d") if price.d
89                                 else None,
90                                 "Instrument": ins_id,
91                                 "Name/Ticker": name,
92                                 "High_Price": price.h,
93                                 "Low_Price": price.l,
94                                 "Closing_Price": price.c,
95                                 "Total_Volume": price.v
96                             })
97
98             return pd.DataFrame(rows)
99
100     def build_factor_dataset(
101         self,
102         target_ids: list[int],
103         kpi_ids: list[int],
104         from_date: datetime,
105         to_date: datetime,
106         report_type: str = "year",
107         count: int = 5
108     ) -> pd.DataFrame:
109
110         prices = self.priser(target_ids=target_ids, from_date=from_date, to_date=to_date)
111
112         kpis = self.fetch_kpi_history(
113             target_ids=target_ids,
114             kpi_ids=kpi_ids,
115             report_type=report_type,

```

```

115         count=count
116     )
117
118     # Pivotera så att varje KPI_ID blir en egen kolumn
119     kpis = kpis.pivot_table(
120         index=["Instrument", "Date", "Name/Ticker", "Period"],
121         columns="KPI_ID",
122         values="KPI"
123     ).reset_index()
124
125     kpis = kpis.rename(columns=self.kpi_dict)
126
127     prices["Instrument"] = prices["Instrument"].astype(str)
128     kpis["Instrument"] = kpis["Instrument"].astype(str)
129
130     prices = prices.sort_values(["Instrument", "Date"])
131     kpis = kpis.sort_values(["Instrument", "Date"])
132
133     prices["Date"] = pd.to_datetime(prices["Date"])
134     kpis["Date"] = pd.to_datetime(kpis["Date"])
135
136     # Förskjut KPI-datum 45 dagar framåt för att undvika lookahead bias
137     kpis["Date"] = kpis["Date"] + pd.DateOffset(days=45)
138
139     prices = prices.sort_values("Date").reset_index(drop=True)
140     kpis = kpis.sort_values("Date").reset_index(drop=True)
141
142     panel = pd.merge_asof(
143         prices,
144         kpis,
145         on="Date",
146         by="Instrument",
147         direction="backward"
148     )
149
150     panel = panel.sort_values(["Instrument", "Date"]).reset_index(drop=True)
151
152     panel["Return"] = panel.groupby("Instrument")["Closing_Price"].pct_change()
153
154     panel = panel.drop(columns=["Name/Ticker_y", "Period"])
155     panel = panel.rename(columns={"Name/Ticker_x": "Name/Ticker"})
156
157     kpi_cols = list(self.kpi_dict.values())
158     panel = panel[
159         ["Date", "Instrument", "Name/Ticker"] + kpi_cols
160         + ["High_Price", "Low_Price", "Closing_Price", "Total_Volume", "Return"]
161     ]
162
163     panel.to_parquet(self.data_dir + "factor_data.parquet", index=False)
164     panel.to_csv(self.data_dir + "factor_data.csv", index=False)
165
166     return panel

```

```

1 class DataCleaner:
2
3     def date_correct_format(self, df: pd.DataFrame) -> pd.DataFrame:
4         #Säkerställer att Date är datetime.
5         df["Date"] = pd.to_datetime(df["Date"])
6         return df
7
8     def sort_df(self, df: pd.DataFrame) -> pd.DataFrame:

```

```

9         #Sorterar så att alla instrumentens rader är i datumordning.
10        return df.sort_values(["Instrument", "Date"]).reset_index(drop=True)
11
12    def remove_dup(self, df: pd.DataFrame) -> pd.DataFrame:
13        #Tar bort rader med samma instrument och datum.
14        return df.drop_duplicates(subset=["Instrument", "Date"])
15
16    def numeric_entries(self, df: pd.DataFrame, cols: list[str]) -> pd.DataFrame:
17        #Konverterar kolumner till numeriska värden; tilldelar NaN vid misslyckande.
18        df = df.copy()
19        df[cols] = df[cols].apply(pd.to_numeric, errors="coerce")
20        return df
21
22    def forward_fill(self, df: pd.DataFrame, factor_cols: list[str]) -> pd.DataFrame:
23
24        #Forward-fyller KPI-värden per instrument.
25        #KPI:er uppdateras årsvis och bör behandlas som en stegfunktion.
26
27        df[factor_cols] = df.groupby("Instrument")[factor_cols].ffill()
28        return df
29
30    def winsorize(
31        self,
32        df: pd.DataFrame,
33        factor_cols: list[str],
34        lower: float = 0.01,
35        upper: float = 0.99
36    ) -> pd.DataFrame:
37
38        df = df.copy()
39        for col in factor_cols:
40            lo = df[col].quantile(lower)
41            hi = df[col].quantile(upper)
42            df[col] = df[col].clip(lo, hi)
43        return df
44
45    def winsorize_series(self, s: pd.Series, lower: float = 0.01, upper: float = 0.99) ->
pd.Series:
46        #Winsoriserar en enskild serie.
47        return s.clip(s.quantile(lower), s.quantile(upper))
48
49    def drop_irrelevant(self, df: pd.DataFrame) -> pd.DataFrame:
50        #Tar bort räprikolumner som är redundanta när Return är beräknad.
51        cols_to_drop = ["High_Price", "Low_Price", "Closing_Price"]
52        return df.drop(columns=[c for c in cols_to_drop if c in df.columns])
53
54    def log_transform(self, df: pd.DataFrame, log_cols: list[str]) -> pd.DataFrame:
55        #Log-transformerar skeva kolumner (t.ex. börsvärde, volym).
56        df = df.copy()
57        existing = [c for c in log_cols if c in df.columns]
58        if existing:
59            df[existing] = df[existing].apply(lambda x: np.log1p(x.clip(lower=0)))
60        return df
61
62    def momentum_factor(self, df: pd.DataFrame, lookback: int = 12, skip: int = 1) -> pd.
DataFrame:
63
64        #Månadsbaserad 12-1 momentum: avkastning från 12 månader sedan till 1 månad sedan.
65
66        df = df.copy()
67        df = df.sort_values(["Instrument", "Date"])

```

```

68     df["Momentum"] = df.groupby("Instrument")["Closing_Price"].transform(
69         lambda x: x.shift(skip) / x.shift(lookback) - 1
70     )
71     return df
72
73 def momentum_12_1_monthly(self, returns: pd.Series) -> pd.Series:
74
75     #Månadsbaserad 12-1 momentum (används i monthly panels):
76
77
78     return (
79         (1 + returns)
80         .shift(1)
81         .rolling(window=11)
82         .apply(np.prod, raw=True) - 1
83     )
84
85 def filter_irregular_reporters(
86     self,
87     df: pd.DataFrame,
88     kpi_cols: list[str],
89     min_gap_days: int = 60
90 ) -> pd.DataFrame:
91
92     #Tar bort instrument där KPI uppdateras snabbare än min_gap_days,
93
94     df = df.copy()
95     bad_instruments = set()
96
97     for col in kpi_cols:
98         for ins_id, group in df.groupby("Instrument"):
99             changed = group[col] != group[col].shift(1)
100             change_dates = group.loc[changed & group[col].notna(), "Date"]
101             if len(change_dates) < 2:
102                 continue
103             gaps = change_dates.diff().dropna().dt.days
104             if (gaps < min_gap_days).any():
105                 bad_instruments.add(ins_id)
106
107     print(f"Tar bort {len(bad_instruments)} instrument med oregelbunden rapportering")
108     return df[~df["Instrument"].isin(bad_instruments)].reset_index(drop=True)
109
110 def clean(self, df: pd.DataFrame, factor_cols: list[str]) -> pd.DataFrame:
111     """
112     #Kör hela rensningspipelinen:
113
114     #Returns
115     #-----
116     #pd.DataFrame
117     #Rensat dataset.
118
119     df = self.date_correct_format(df)
120     df = self.sort_df(df)
121     df = self.remove_dup(df)
122     if "Closing_Price" in df.columns:
123         df = self.momentum_factor(df)
124     df = self.filter_irregular_reporters(df, factor_cols, min_gap_days=60)
125     df = self.numeric_entries(df, factor_cols)
126     df = self.forward_fill(df, factor_cols)
127     df = self.winsorize(df, factor_cols)
128     df = self.log_transform(df, ["Borsvarde", "Total_Volume"]) # ignoreras om

```

```

129     kolumner saknas
130     df = self.drop_irrelevant(df)
131     df = self.winsorize(df, ["Return"], lower=0.01, upper=0.99)
132     return df
133
134 def data_report(self, df: pd.DataFrame) -> dict:
135     #Skriver ut en strukturerad datakvalitetsrapport.
136     numeric_df = df.select_dtypes(include="number")
137     z = (numeric_df - numeric_df.mean()) / numeric_df.std()
138
139     report = {
140         "missing_values": df.isna().mean().sort_values(ascending=False),
141         "outlier_count": (z.abs() > 5).sum().sort_values(ascending=False),
142         "time_order_status": df.groupby("Instrument")["Date"].apply(
143             lambda x: x.is_monotonic_increasing
144         ),
145         "duplicates": df.duplicated().sum(),
146         "date_range": (df["Date"].min(), df["Date"].max()),
147         "obs_per_instrument": df.groupby("Instrument").size().describe(),
148         "obs_per_instrument_list": df.groupby(["Instrument", "Name/Ticker"]).size(),
149     }
150
151     for key, value in report.items():
152         print(f"\n{'='*40}")
153         print(f"    {key.upper()}")
154         print(f"{'='*40}")
155         print(value)
156
157     return report

```

```

1 class Factor_Engineering:
2     """
3     Innehåller all faktor-engineering, regression, kovariansmodellering
4     och portföljoptimering.
5     """
6
7     def rank_standardize(self, df: pd.DataFrame, cols: list[str]) -> pd.DataFrame:
8         """
9         Korsektionell rankstandardisering per datum: span [-1, 1].
10        """
11        df = df.copy()
12        df[cols] = df.groupby("Date")[cols].transform(lambda x: x.rank(pct=True))
13        df[cols] = 2 * df[cols] - 1
14        return df
15
16    def ema(self, df: pd.DataFrame, cols: list[str], span: int) -> pd.DataFrame:
17        """
18        Exponentiell viktning av faktorvärden per instrument.
19        """
20        df = df.copy()
21        df[cols] = df.groupby("Instrument")[cols].transform(
22            lambda x: x.ewm(span=span).mean()
23        )
24        return df
25
26    def forward_return(self, df: pd.DataFrame, horizon: int) -> pd.DataFrame:
27        """
28        Skapar Forward_return: summan av avkastning de kommande 'horizon' stegen.
29
30        Parameters
31        -----

```

```

32     horizon : Antal steg framåt
33     """
34     df = df.copy()
35     df["Forward_return"] = df.groupby("Instrument")["Return"].transform(
36         lambda x: x.shift(-horizon).rolling(horizon).sum()
37     )
38     return df
39
40 def fama_french_regression(
41     self,
42     df: pd.DataFrame,
43     cols: list[str]
44 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.DataFrame]:
45     """
46     Kör Fama-French tvärsnittssregression (OLS) per månad.
47
48     Returns
49     -----
50     results      : DataFrame med mean_beta, std_beta, t_stat, significans per faktor
51     betas_df     : Tidserie av faktorpriser (lambda_t) per datum
52     idiosyn_var  : Idiosynkratisk varians per instrument (från residualer)
53     r2_df        : R per datum
54     """
55     df = df.copy()
56     df[cols] = df.groupby("Date")[cols].transform(
57         lambda x: (x - x.mean()) / (x.std(ddof=0) + 1e-8))
58     df["Date"] = pd.to_datetime(df["Date"])
59
60     lambda_t      = []
61     r2_list       = []
62     date_lst      = []
63     residuals_dict = {}
64
65     monthly_dates = (
66         df.groupby(df["Date"].dt.to_period("M"))["Date"]
67         .first().values
68     )
69
70     for date in monthly_dates:
71         df_slice = (
72             df[df["Date"] == date]
73             .replace([np.inf, -np.inf], np.nan)
74             .dropna(subset=["Forward_return"] + cols)
75         )
76
77         if len(df_slice) < len(cols) + 2:
78             continue
79
80         X      = sm.add_constant(df_slice[cols])
81         y      = df_slice["Forward_return"]
82         model = sm.OLS(y, X).fit()
83
84         lambda_t.append(pd.Series(model.params, name=date))
85         r2_list.append(model.rsquared)
86         date_lst.append(date)
87
88         resid_series = pd.Series(model.resid.values, index=df_slice["Instrument"].
values)
89         for ins, resid in resid_series.items():
90             residuals_dict.setdefault(ins, []).append(resid)
91

```

```

92     betas_df_with_const = pd.DataFrame(lambda_t)
93
94
95     T = len(betas_df)
96     results = pd.DataFrame({
97         "mean_beta": betas_df.mean(),
98         "std_beta": betas_df.std(),
99         "t_stat": betas_df.mean() / (betas_df.std() / np.sqrt(T)),
100    })
101    results["significans"] = results["t_stat"].abs() > 1.96
102    results.index = pd.Index(cols, name="Factor")
103
104    r2_df = pd.DataFrame({"Date": date_lst, "R_squared": r2_list})
105
106    if not residuals_dict:
107        raise ValueError(
108            "residuals_dict är tom inga residualer sparades. "
109            "Kontrollera att Forward_return har icke-NaN-värden och att "
110            "tillräckligt många datum passerar minobservationsfiltret."
111        )
112
113    idiosyn_var = pd.Series(
114        {ins: np.var(resids) for ins, resids in residuals_dict.items()}
115    )
116
117    return results, betas_df, idiosyn_var, r2_df
118
119    def build_sector_factor_model(
120        self,
121        train_sector_data: pd.DataFrame,
122        train_periods: list,
123        features: list[str],
124        min_obs_per_cross_section: int,
125        min_valid_months: int,
126        use_intercept: bool = True
127    ) -> tuple:
128        """
129        Skattar sektorvis faktorstruktur från träningsfönstret via OLS per
130        korsektionsdatum.
131
132        Returns
133        -----
134        F_sector : Kovariansmatris för faktorpriser (K x K)
135        lambda_mean : Medelvärde av faktorpriser över tid
136        intercept_mean: Medelvärde av intercept
137        epsilon_var : Residualvarians per aktie
138        """
139        lambdas = []
140        residuals = []
141        intercepts = []
142
143        for period in train_periods:
144            slice_df = train_sector_data[train_sector_data["Period"] == period].copy()
145
146            if len(slice_df) < min_obs_per_cross_section:
147                continue
148
149            X = slice_df[features]
150            y = slice_df["Target_Return"]
151
152            model = LinearRegression(fit_intercept=use_intercept)

```

```

152         model.fit(X, y)
153
154         lambdas.append(model.coef_)
155         intercepts.append(model.intercept_ if use_intercept else 0.0)
156
157         resid = y - model.predict(X)
158         residuals.append(pd.DataFrame({
159             "ins_id": slice_df["ins_id"].values,
160             "resid": resid.values
161         }))
162
163         if len(lambdas) < min_valid_months:
164             return None, None, None, None
165
166         lambdas = np.asarray(lambdas)
167         F_sector = np.cov(lambdas.T, ddof=1)
168
169         if np.ndim(F_sector) == 0:
170             return None, None, None, None
171
172         lambda_mean = np.mean(lambdas, axis=0)
173         intercept_mean = float(np.mean(intercepts))
174
175         if not residuals:
176             return None, None, None, None
177
178         residuals_df = pd.concat(residuals, ignore_index=True)
179         epsilon_var = residuals_df.groupby("ins_id")["resid"].var()
180
181         return F_sector, lambda_mean, intercept_mean, epsilon_var
182
183 def mu_estimate(self, df: pd.DataFrame, cols: list[str], lambda_weights: pd.Series) ->
    pd.Series:
184     """
185     Skattar förväntad avkastning:  $\mu_i = \sum_k (\beta_{ik} * \lambda_k)$ .
186
187     Parameters
188     -----
189     df           : Senaste tillgängliga data per instrument
190     cols         : Faktorkolumner
191     lambda_weights : Genomsnittliga faktorpriser ( $\lambda_{\hat{}}$ )
192     """
193     df = df.copy()
194     mu = df[cols].mul(lambda_weights).sum(axis=1)
195     mu.index = df["Instrument"].values
196     return mu
197
198 def build_covariance_matrix(
199     self,
200     B: pd.DataFrame,
201     cov_F: pd.DataFrame,
202     epsilon: pd.Series,
203     ridge: float = 1e-6
204 ) -> pd.DataFrame:
205     """
206     Bygger kovariansmatrisen via Barra-stil faktormodell:
207          $\Sigma = B @ cov_F @ B.T + D$ 
208
209     Parameters
210     -----
211     B           : Faktorexponeringsmatris (N x K)

```

```

212     cov_F : Faktorkövariansmatris (K x K), LW-krympt
213     epsilon : Idiosynkratisk varians per instrument
214     ridge : Regulariseringsterm för numerisk stabilitet
215
216     Returns
217     -----
218     pd.DataFrame (N x N)
219     """
220     systematic = B.values @ cov_F.values @ B.values.T
221     epsilon_filled = epsilon.reindex(B.index).fillna(epsilon.median())
222     idiosyncratic = np.diag(epsilon_filled.to_numpy(dtype=float))
223     Sigma = systematic + idiosyncratic + np.eye(len(B)) * ridge
224     return pd.DataFrame(Sigma, index=B.index, columns=B.index)
225
226     def build_stock_covariance_from_arrays(
227         self,
228         B: pd.DataFrame,
229         F_sector: np.ndarray,
230         epsilon_series: pd.Series,
231         ridge: float = 1e-4
232     ) -> np.ndarray:
233         """
234         Variant av build_covariance_matrix som arbetar med numpy-arrayer
235         (används i sektorvis optimering).
236
237         Returns
238         -----
239         np.ndarray (N x N)
240         """
241         instruments = B.index
242         epsilon = epsilon_series.reindex(instruments)
243         median_epsilon = epsilon.median()
244         if pd.isna(median_epsilon) or median_epsilon <= 0:
245             median_epsilon = 1e-4
246         epsilon = np.maximum(epsilon.fillna(median_epsilon).values, 1e-6)
247         D = np.diag(epsilon)
248         Sigma = B.values @ F_sector @ B.values.T + D + np.eye(len(B)) * ridge
249         return Sigma
250
251     @staticmethod
252     def project_to_capped_simplex(
253         raw_weights: np.ndarray,
254         max_weight: float,
255         tol: float = 1e-12
256     ):
257         """
258         Projicerar en viktvektor till long-only simplexen med vikttak:
259         - vikter >= 0
260         - vikter <= max_weight
261         - summa = 1
262
263         Använder iterativ fördelning av överskottsmassa.
264         Returnerar None om problemet är olösbart. Används i sektormodellen.
265         """
266         w = np.asarray(raw_weights, dtype=float).copy()
267         w = np.maximum(w, 0.0)
268         n = len(w)
269         if n == 0 or max_weight * n < 1 - tol:
270             return None
271
272         w = np.ones(n) / n if w.sum() <= tol else w / w.sum()

```

```

273
274 free = np.ones(n, dtype=bool)
275 fixed = np.zeros(n, dtype=bool)
276 out = np.zeros(n)
277
278 while True:
279     if free.sum() == 0:
280         break
281     remaining_mass = 1.0 - out[fixed].sum()
282     if remaining_mass < -tol:
283         return None
284
285     fw = w[free].copy()
286     fw = np.ones(free.sum()) / free.sum() if fw.sum() <= tol else fw / fw.sum()
287     candidate = fw * remaining_mass
288     over_cap = candidate > max_weight + tol
289
290     if not np.any(over_cap):
291         out[free] = candidate
292         break
293
294     free_idx = np.where(free)[0]
295     capped_idx = free_idx[over_cap]
296     out[capped_idx] = max_weight
297     fixed[capped_idx] = True
298     free[capped_idx] = False
299     w[capped_idx] = 0.0
300
301 out = np.maximum(out, 0.0)
302 if out.sum() <= tol or np.any(out > max_weight + 1e-8):
303     return None
304 return out / out.sum()
305
306 def _objective(self, w, co_var, mu, theta, pi_prev=None, gamma=None):
307     """
308     Markowitzs målfunktion med valfri transaktionskostnadsterm.
309
310     maximize: w'μ - θ * w'Sigma w - γ * |w - w_prev|
311     """
312     utility = w.T @ mu - theta * (w.T @ co_var @ w)
313     if pi_prev is not None and gamma is not None:
314         utility -= np.sum(gamma * np.abs(w - pi_prev))
315     return -utility
316
317 def markowitz(
318     self,
319     mu: pd.Series,
320     co_var: pd.DataFrame,
321     theta: float,
322     max_weight: float = 0.05,
323     pi_prev: np.ndarray | None = None
324 ) -> pd.Series:
325     """
326     Markowitz mean-variansoptimering (SLSQP).
327     Stöder valfri transaktionskostnadsterm om pi_prev anges.
328
329     Parameters
330     -----
331     mu : Förväntad avkastning per instrument
332     co_var : Kovariansmatris (N x N)
333     theta : Riskaversionsparameter

```

```

334     max_weight : Maximal vikt per instrument
335     pi_prev    : Föregående periods vikter (för transaktionskostnad)
336
337     Returns
338     -----
339     pd.Series Optimerade portföljvikter
340     """
341     mu = mu.reindex(co_var.index)
342     N = len(mu)
343     w0 = np.ones(N) / N
344
345     gamma = None
346     if pi_prev is not None:
347         sigma_diag = np.sqrt(np.diag(co_var.values))
348         gamma = 0.05 * sigma_diag
349         if len(pi_prev) != N:
350             pi_prev = np.zeros(N)
351
352     constraints = {"type": "eq", "fun": lambda w: np.sum(w) - 1}
353     bounds = [(0, max_weight)] * N
354
355     result = minimize(
356         self._objective,
357         w0,
358         args=(co_var.values, mu.values, theta, pi_prev, gamma),
359         method="SLSQP",
360         constraints=constraints,
361         bounds=bounds
362     )
363
364     weights = pd.Series(result.x, index=mu.index)
365     if not result.success:
366         print(result.message)
367
368     weights = weights / weights.sum()
369     return weights
370
371 def solve_markowitz_analytical(
372     self,
373     B: pd.DataFrame,
374     mu: np.ndarray,
375     F_sector: np.ndarray,
376     epsilon_series: pd.Series,
377     theta: float,
378     r: float,
379     max_weight: float
380 ) -> tuple:
381     """
382     Analytisk Markowitz-lösning (pseudoinvers):
383         w_raw = inv(theta * Sigma) @ (mu - r)
384
385     Projicerar sedan vikterna till simplexen med cap.
386     Snabbare och mer stabil än SLSQP för sektorvis optimering.
387
388     Returns
389     -----
390     (weights, Sigma) eller (None, Sigma) vid misslyckande
391     """
392     Sigma = self.build_stock_covariance_from_arrays(B, F_sector, epsilon_series)
393     adjusted_mu = np.asarray(mu, dtype=float) - r
394

```

```

395         try:
396             raw_weights = np.linalg.pinv(theta * Sigma) @ adjusted_mu
397         except Exception:
398             return None, None
399
400         weights = self.project_to_capped_simplex(raw_weights, max_weight=max_weight)
401         return weights, Sigma
402
403     @staticmethod
404     def compute_turnover(
405         prev_weights: pd.Series | None,
406         prev_realized: pd.Series | None,
407         curr_weights: pd.Series
408     ) -> float:
409         """
410         Turnover = 0.5 * sum(|w_curr - w_prev_drift|).
411         Föregående portfölj tillåts driva med realiserad avkastning
412         innan jämförelse görs.
413         """
414         if prev_weights is None or prev_realized is None:
415             return np.nan
416
417         all_instruments = curr_weights.index.union(prev_weights.index)
418         w_prev = prev_weights.reindex(all_instruments).fillna(0.0)
419         r_prev = prev_realized.reindex(all_instruments).fillna(0.0)
420         w_curr = curr_weights.reindex(all_instruments).fillna(0.0)
421
422         w_prev_drift = w_prev * (1.0 + r_prev)
423         if w_prev_drift.sum() > 0:
424             w_prev_drift /= w_prev_drift.sum()
425
426         return 0.5 * np.sum(np.abs(w_curr - w_prev_drift))
427
428     def forward_run(
429         self,
430         df: pd.DataFrame,
431         factor_cols: list[str],
432         span: int,
433         horizon: int,
434         theta: float,
435         max_weight: float = 0.10,
436         min_weight: float = 0.005
437     ) -> tuple:
438         df = df.copy()
439         df["Date"] = pd.to_datetime(df["Date"])
440         df = self.ema(df, cols=factor_cols, span=span)
441         df = df.replace([np.inf, -np.inf], np.nan)
442
443         if "Forward_return" not in df.columns:
444             df = self.forward_return(df, horizon=horizon)
445
446
447         df_latest = df.sort_values("Date").groupby("Instrument").last()
448         B = df_latest[factor_cols].dropna()
449
450         # z-score separat kopia för regressionen
451         df_reg = self.zscore_standardize(df.copy(), cols=factor_cols)
452         df_reg = df_reg.replace([np.inf, -np.inf], np.nan)
453
454         results, betas_df, idiosyn_var, r2_df = self.fama_french_regression(df=df_reg,
455                                     cols=factor_cols)

```

```

455
456     # Ledoit-Wolf-krympning
457     numeric_betas = betas_df.select_dtypes(include=[np.number]).dropna()
458     lw             = LedoitWolf().fit(numeric_betas.values)
459     cov_F_shrunk   = pd.DataFrame(lw.covariance_, index=numeric_betas.columns, columns=
numeric_betas.columns)

460
461     # Filtrera till instrument med idiosynkratisk varians
462     common = B.index.intersection(idiosyn_var.index)
463     B      = B.loc[common]
464
465     mu      = self.mu_estimate(df_latest.reset_index(), factor_cols, betas_df.
mean())
466     mu      = mu.reindex(common).dropna()
467     B      = B.loc[mu.index]
468
469     co_var_matrix = self.build_covariance_matrix(B=B, cov_F=cov_F_shrunk, epsilon=
idiosyn_var)
470     portfolio_weights = self.markowitz(mu=mu, co_var=co_var_matrix, theta=theta,
max_weight=max_weight)
471
472     portfolio_weights = portfolio_weights[portfolio_weights >= min_weight]
473     portfolio_weights = portfolio_weights / portfolio_weights.sum()
474
475     return portfolio_weights, results, co_var_matrix, betas_df, r2_df
476
477 def backtesting(
478     self,
479     df: pd.DataFrame,
480     factor_cols: list[str],
481     span: int,
482     horizon: int,
483     theta: float,
484     max_weight: float = 0.05,
485     min_weight: float = 0.0167,
486     min_periods: int = 12
487 ) -> tuple:
488     """
489     Rullande månadsbaserad backtesting.
490     Rebalanserar portföljen varje månad baserat på data tillgänglig
491     t.o.m. rebalanseringsdatumet.
492
493     Parameters
494     -----
495     min_periods : Burn-in-period (antal månader innan första optimering)
496
497     Returns
498     -----
499     (df_return, total_return, fam_results, weights_df, cov_F, betas_df, r2_df)
500     """
501     returns = []
502     weights_history = []
503     weights_dates = []
504
505     df = df.copy()
506     df["Date"] = pd.to_datetime(df["Date"])
507     df = self.rank_standardize(df=df, cols=factor_cols)
508     df = self.forward_return(df, horizon=horizon)
509
510     rebalancing_dates = (
511         df.groupby(df["Date"].dt.to_period("M"))["Date"]

```

```

512         .first().values
513     )
514
515     for i in range(len(rebalancing_dates) - 1):
516         start_date = rebalancing_dates[i]
517         end_date = rebalancing_dates[i + 1]
518
519         df_available = df[df["Date"] <= start_date]
520         months_available = df_available["Date"].dt.to_period("M").nunique()
521         if months_available < min_periods:
522             continue
523
524         n_instruments = df_available["Instrument"].nunique()
525         n_valid_fr = df_available.dropna(subset=["Forward_return"])["Instrument"].
nunique()
526
527         portfolio_weights, fam_results, cov_F, betas_df, r2_df = self.forward_run(
528             df=df_available,
529             factor_cols=factor_cols,
530             span=span,
531             horizon=horizon,
532             theta=theta,
533             max_weight=max_weight,
534             min_weight=min_weight
535         )
536
537         weights_history.append(portfolio_weights)
538         weights_dates.append(str(start_date))
539
540         # Geometrisk sammansatt avkastning för perioden
541         return_period = (
542             df[(df["Date"] >= start_date) & (df["Date"] < end_date)]
543             .groupby("Instrument")["Return"]
544             .apply(lambda x: (1 + x).prod() - 1)
545         )
546
547         common = pd.Index(portfolio_weights.index).intersection(return_period.index
)
548
549         w_common = portfolio_weights[common]
550         w_common = w_common / w_common.sum()
551         R_portfolio = w_common @ return_period[common]
552
553         returns.append({"Date": end_date, "Return": R_portfolio})
554
555         df_return = pd.DataFrame(returns).set_index("Date")
556         total_return = (1 + df_return["Return"]).cumprod()
557
558         weights_df = pd.concat(weights_history, axis=0, keys=weights_dates)
559         weights_df = weights_df.unstack(level=1).fillna(0)
560
561         return df_return, total_return, fam_results, weights_df, cov_F, betas_df, r2_df
562
563     def backtesting_sector(
564         self,
565         df: pd.DataFrame,
566         features: list[str],
567         sector_col: str = "Sector",
568         train_window: int = 24,
569         theta: float = 3.0,
570         r: float = 0.02 / 12,
571         max_weight: float = 0.40,

```

```

571     top_pct:                                float = 0.06,
572     min_valid_months:                       int    = 6,
573     min_names_for_selection:                int    = 8,
574     min_selected_per_sector:                int    = 0,
575     max_selected_per_sector:                int    = 4,
576     min_obs_per_cross_section:              int | None = None,
577     debug:                                  bool    = False
578 ) -> dict:
579     """
580     Sektorvis Fama-French + Markowitz backtesting (analytisk lösning).
581
582     Flöde per testperiod:
583     1. Kör OLS-regression per sektor på träningsfönstret -> faktorpriser + kovarians
584     2. Predicera  $\mu_i = B_i @ \lambda_{\text{mean}}$  per instrument i testsektorn
585     3. Välj topp-N instrument per sektor (top_pct)
586     4. Lös Markowitz analytiskt per sektor
587     5. Kombinera sektorvikter proportionellt mot sektorns genomsnittliga  $\mu$ 
588
589     Parameters
590     -----
591     df                : Paneldata med kolumner Period, ins_id, Sector, Target_Return,
features
592     features          : Faktorskolumner
593     sector_col        : Kolumnnamn för sektor
594     train_window      : Antal månader i träningsfönstret
595     theta             : Riskaversion
596     r                 : Riskfri månadsränta
597     max_weight        : Maxvikt per aktie inom sektorportfölj
598     top_pct           : Andel aktier som väljs per sektor
599
600     Returns
601     -----
602     dict med nycklar:
603         portfolio_returns_mw : pd.Series
604         portfolio_returns_eq : pd.Series (lika-viktad benchmark)
605         benchmark_returns    : pd.Series (equal-weight universum)
606         weights_mw_df        : pd.DataFrame
607         diagnostics          : dict med HHI, ENB, turnover m.m.
608     """
609     if min_obs_per_cross_section is None:
610         min_obs_per_cross_section = len(features) + 5
611
612     df = df.copy()
613     df["Period"] = pd.to_datetime(df["Period"])
614
615     periods      = sorted(df["Period"].dropna().unique())
616     all_sectors  = sorted(df[sector_col].dropna().unique())
617
618     portfolio_returns_mw = []
619     portfolio_returns_eq = []
620     benchmark_returns    = []
621     dates                = []
622     all_weights_mw       = []
623     all_weights_eq       = []
624     turnovers_mw         = []
625     turnovers_eq         = []
626     prev_weights_mw      = None
627     prev_weights_eq      = None
628     prev_realized        = None
629     n_holdings_list      = []
630     hhi_list             = []

```

```

631         enb_list = []
632
633     for i in range(train_window, len( periods )):
634         current_test_period = periods[i]
635         train_periods = periods[i - train_window:i]
636
637         train_data_all = df[df["Period"].isin(train_periods)].dropna(
638             subset=features + ["Target_Return"]
639         ).copy()
640
641         test_data_all = df[df["Period"] == current_test_period].dropna(
642             subset=features + ["Target_Return"]
643         ).copy()
644
645         if train_data_all.empty or test_data_all.empty:
646             continue
647
648         sector_selected_map = {}
649
650         for sector in all_sectors:
651             train_sector = train_data_all[train_data_all[sector_col] == sector].copy()
652             test_sector = test_data_all[test_data_all[sector_col] == sector].copy()
653
654             if train_sector.empty or test_sector.empty:
655                 continue
656             if len(test_sector) < min_names_for_selection:
657                 continue
658
659             F_sector, lambda_mean, intercept_mean, epsilon_series = self.
build_sector_factor_model(
660                 train_sector_data=train_sector,
661                 train_periods=train_periods,
662                 features=features,
663                 min_obs_per_cross_section=min_obs_per_cross_section,
664                 min_valid_months=min_valid_months,
665                 use_intercept=True
666             )
667
668             if F_sector is None:
669                 continue
670
671             B_full = test_sector.set_index("ins_id")[features]
672             mu_raw = intercept_mean + B_full.values @ lambda_mean
673             mu_full = pd.Series(mu_raw, index=B_full.index)
674
675             n_select = int(np.ceil(len(mu_full) * top_pct))
676             n_select = max(min_selected_per_sector, n_select)
677             n_select = min(max_selected_per_sector, n_select, len(mu_full))
678
679             selected_instruments = mu_full.nlargest(n_select).index
680             if len(selected_instruments) == 0:
681                 continue
682
683             B = B_full.loc[selected_instruments]
684             mu = mu_full.loc[selected_instruments].values
685             mu_series = pd.Series(mu, index=selected_instruments)
686
687             weights_sector_mw, cov_sector = self.solve_markowitz_analytical(
688                 B=B, mu=mu, F_sector=F_sector,
689                 epsilon_series=epsilon_series,
690                 theta=theta, r=r, max_weight=max_weight

```

```

691         )
692
693         if weights_sector_mw is None:
694             if debug:
695                 print(f"Inga giltiga MW-vikter för sektor {sector}, period {
current_test_period}")
696                 continue
697
698         weights_sector_eq = np.repeat(1 / len(selected_instruments), len(
selected_instruments))
699
700         sector_selected_map[sector] = {
701             "instruments": selected_instruments,
702             "weights_mw": pd.Series(weights_sector_mw, index=selected_instruments
),
703             "weights_eq": pd.Series(weights_sector_eq, index=selected_instruments
),
704             "mu": mu_series,
705             "cov": cov_sector
706         }
707
708         if not sector_selected_map:
709             continue
710
711         # Kombinera sektorportföljer proportionellt mot sektorns mu
712         active_sectors = sorted(sector_selected_map.keys())
713         sector_score = pd.Series(
714             {s: sector_selected_map[s]["mu"].mean() for s in active_sectors},
715             dtype=float
716         ).clip(lower=0)
717
718         sector_alloc_mw = sector_score / sector_score.sum() if sector_score.sum() > 0
\
719         else pd.Series(1 / len(active_sectors), index=active_sectors)
720         sector_alloc_eq = pd.Series(1 / len(active_sectors), index=active_sectors)
721
722         w_curr_mw = pd.concat(
723             [sector_selected_map[s]["weights_mw"] * sector_alloc_mw[s] for s in
active_sectors]
724         ).groupby(level=0).sum()
725
726         w_curr_eq = pd.concat(
727             [sector_selected_map[s]["weights_eq"] * sector_alloc_eq[s] for s in
active_sectors]
728         ).groupby(level=0).sum()
729
730         if w_curr_mw.sum() > 0:
731             w_curr_mw /= w_curr_mw.sum()
732         if w_curr_eq.sum() > 0:
733             w_curr_eq /= w_curr_eq.sum()
734
735         selected_total = w_curr_mw.index.union(w_curr_eq.index)
736         realized_series = (
737             test_data_all.set_index("ins_id")
738             .reindex(selected_total)["Target_Return"]
739             .fillna(0.0)
740         )
741
742         # Diagnostik
743         hhi = np.sum(w_curr_mw.values ** 2)
744         hhi_list.append(hhi)

```

```

745         enb_list.append(1 / hhi if hhi > 0 else np.nan)
746         n_holdings_list.append((w_curr_mw > 0).sum())
747
748         turnover_mw = self.compute_turnover(prev_weights_mw, prev_realized, w_curr_mw)
749         turnover_eq = self.compute_turnover(prev_weights_eq, prev_realized, w_curr_eq)
750         if pd.notna(turnover_mw):
751             turnovers_mw.append(turnover_mw)
752         if pd.notna(turnover_eq):
753             turnovers_eq.append(turnover_eq)
754
755         prev_weights_mw = w_curr_mw.copy()
756         prev_weights_eq = w_curr_eq.copy()
757         prev_realized = realized_series.copy()
758
759         all_weights_mw.append(w_curr_mw.rename(current_test_period))
760         all_weights_eq.append(w_curr_eq.rename(current_test_period))
761
762         portfolio_returns_mw.append((w_curr_mw * realized_series.reindex(w_curr_mw.
index)).sum())
763         portfolio_returns_eq.append((w_curr_eq * realized_series.reindex(w_curr_eq.
index)).sum())
764         benchmark_returns.append(test_data_all["Target_Return"].mean())
765         dates.append(current_test_period)
766
767         return {
768             "portfolio_returns_mw": pd.Series(portfolio_returns_mw, index=dates, name="
Markowitz"),
769             "portfolio_returns_eq": pd.Series(portfolio_returns_eq, index=dates, name="
EqualWeightSameStocks"),
770             "benchmark_returns": pd.Series(benchmark_returns, index=dates, name="
Benchmark"),
771             "weights_mw_df": pd.concat(all_weights_mw, axis=1).T if all_weights_mw
else pd.DataFrame(),
772             "weights_eq_df": pd.concat(all_weights_eq, axis=1).T if all_weights_eq
else pd.DataFrame(),
773             "diagnostics": {
774                 "avg_turnover_mw": np.mean(turnovers_mw) if turnovers_mw else np.nan,
775                 "avg_turnover_eq": np.mean(turnovers_eq) if turnovers_eq else np.nan,
776                 "n_holdings_series": pd.Series(n_holdings_list, index=dates, name="
N_Holdings"),
777                 "hhi_series": pd.Series(hhi_list, index=dates, name="HHI"),
778                 "enb_series": pd.Series(enb_list, index=dates, name="
Effective_N"),
779             }
780         }

```

```

1  class PerformanceEval:
2      """
3      Beräknar och rapporterar standardiserade prestandamått
4      för en portföljserie relativt ett benchmark.
5      """
6
7      def __init__(
8          self,
9          returns: pd.DataFrame,
10         benchmark: pd.DataFrame,
11         rf: float,
12         freq: int,
13         results: pd.DataFrame,
14         cov_matrix: pd.DataFrame,
15         betas_df: pd.DataFrame

```

```

16 ) -> None:
17     self.returns = returns.copy()
18     self.benchmark = benchmark.copy()
19
20     returns.index = pd.to_datetime(returns.index.astype(str)).to_period("M")
21     benchmark.index = pd.to_datetime(benchmark.index.astype(str)).to_period("M")
22     returns.index = returns.index - 1
23     self.returns, self.benchmark = returns.align(benchmark, join="inner")
24
25     self.rf = rf
26     self.freq = freq
27     self.results = results
28     self.cov_matrix = cov_matrix
29     self.betas_df = betas_df
30
31 def cumulative_return(self) -> pd.Series:
32     """Total avkastning över hela perioden."""
33     return (1 + self.returns["Return"]).cumprod() - 1
34
35 def annualized_return(self) -> float:
36     """Genomsnittlig årsavkastning."""
37     total_periods = len(self.returns)
38     cumulative = self.cumulative_return().iloc[-1] + 1
39     return cumulative ** (self.freq / total_periods) - 1
40
41 def annualized_volatility(self) -> float:
42     """Annualiserad volatilitet."""
43     return self.returns["Return"].std() * np.sqrt(self.freq)
44
45 def sharpe_ratio(self) -> float:
46     """Sharpe-kvot"""
47     return (self.annualized_return() - self.rf) / self.annualized_volatility()
48
49 def sortino_ratio(self, target_return: float = 0) -> float:
50     """Sortino-kvot: Sharpe men mäter bara nedsidesrisk."""
51     excess_return = self.returns["Return"] - target_return
52     negative_return = self.returns["Return"][excess_return < 0]
53     negative_std = negative_return.std() * np.sqrt(self.freq)
54     return (self.annualized_return() - target_return) / negative_std
55
56 def max_drawdown(self) -> float:
57     """
58     Maximalt kontinuerligt värdetapp.
59     Betraktas ofta som ett panikindex för investeraren.
60     """
61     wealth = self.cumulative_return() + 1
62     max_wealth = wealth.cummax()
63     return (wealth / max_wealth - 1).min()
64
65 def tracking_error(self) -> float:
66     """
67     Standardavvikelse för aktiv avkastning (portfölj minus benchmark).
68     Lågt värde passiv förvaltning, högt aktiv.
69     """
70     active_return = self.returns["Return"] - self.benchmark["Return"]
71     return active_return.std() * np.sqrt(self.freq)
72
73 def information_ratio(self) -> float:
74     """Sharpe-kvot för aktiv avkastning. Över 0.5 anses generellt bra."""
75     mean_active = (self.returns["Return"] - self.benchmark["Return"]).mean()
76     return mean_active / self.tracking_error()

```

```

77
78 def hit_rate(self) -> float:
79     """Andel perioder där portföljen slog benchmark."""
80     active = self.returns["Return"] - self.benchmark["Return"]
81     return (active > 0).mean()
82
83 def win_loss_ratio(self) -> float:
84     """
85     Kvot mellan genomsnittlig vinnarmånad och genomsnittlig förlustemånad.
86     Över 1.0 innebär att vinsterna är större än förlusterna i genomsnitt.
87     """
88     active = self.returns["Return"] - self.benchmark["Return"]
89     avg_win = active[active > 0].mean()
90     avg_loss = active[active < 0].abs().mean()
91     return avg_win / avg_loss
92
93 def herfindahl_index(self, weights: pd.Series) -> float:
94     """Viktkoncentrationsindex. 0 = perfekt diversifierad, 1 = enstaka aktie."""
95     return (weights ** 2).sum()
96
97 def effective_n_bets(self, weights: pd.Series) -> float:
98     """Effektivt antal bets (1 / HHI)."""
99     hhi = self.herfindahl_index(weights)
100    return 1 / hhi if hhi > 0 else np.nan
101
102 def ledoit_wolf_shrinkage_intensity(self, betas_df: pd.DataFrame) -> float:
103     """
104     Rapporterar LW-krympningskoefficienten.
105     Nära 0 = sampelkovariansen är tillförlitlig.
106     Nära 1 = sampelkovariansen är nästan oanvändbar.
107     """
108     numeric_cols = betas_df.select_dtypes(include=[np.number]).columns
109     values = betas_df[numeric_cols].dropna().values.astype(float)
110
111     if values.shape[0] < 2:
112         print(" [!] För få observationer för LW-intensitetsestimering")
113         return np.nan
114     if values.shape[0] < values.shape[1] + 1:
115         print(f" [!] För få rader ({values.shape[0]}) vs kolumner ({values.shape[1]})
116         för LW")
117         return np.nan
118
119     return LedoitWolf().fit(values).shrinkage_
120
121 def capm_regression_metrics(self) -> dict:
122     """
123     CAPM-regression för portföljens rörelse relativt marknaden.
124
125     Returns
126     -----
127     dict med Alpha, Beta, t_stats, p_values, r2
128     """
129     port_returns = self.returns["Return"].copy()
130     bench_returns = self.benchmark["Return"].copy()
131
132     if hasattr(port_returns.index, "to_timestamp"):
133         port_returns.index = port_returns.index.to_timestamp()
134     if hasattr(bench_returns.index, "to_timestamp"):
135         bench_returns.index = bench_returns.index.to_timestamp()
136
137     port_returns, bench_returns = port_returns.align(bench_returns, join="inner")

```

```

137     print(f"Överlappande perioder: {len(port_returns)}")
138
139     y = port_returns - (self.rf / self.freq)
140     X = bench_returns - (self.rf / self.freq)
141     X = sm.add_constant(X.to_frame(name="Benchmark"))
142
143     model = sm.OLS(y, X).fit()
144
145     return {
146         "Alpha":    model.params["const"],
147         "Beta":     model.params["Benchmark"],
148         "t_stats":  model.tvalues,
149         "p_values": model.pvalues,
150         "r2":       model.rsquared
151     }
152
153     def _report_cov_diagnostics(self) -> None:
154         """
155         Diagnostiserar kovariansmatrisen: konditionsnummer, egenvärden,
156         rang, symmetri och diagonalhälsa.
157         """
158         cov = self.cov_matrix.values
159         n    = cov.shape[0]
160
161         sym_error    = np.max(np.abs(cov - cov.T))
162         is_symmetric = sym_error < 1e-8
163         print(f"   Shape:           {n} x {n}")
164         print(f"   Symmetric:         {'Yes' if is_symmetric else f'NO (max asymmetry: {sym_error:.2e}')}")
165
166         diag        = np.diag(cov)
167         n_zero_var   = np.sum(diag < 1e-10)
168         n_neg_var    = np.sum(diag < 0)
169         print(f"   Min Variance:       {diag.min():.4e}")
170         print(f"   Max Variance:       {diag.max():.4e}")
171         print(f"   Near-Zero Variances: {n_zero_var} (< 1e-10)")
172         print(f"   Negative Variances:  {n_neg_var}")
173
174         n_neg_eig = 0
175         try:
176             eigvals    = np.linalg.eigvalsh(cov)
177             min_eigval = eigvals.min()
178             max_eigval = eigvals.max()
179             n_neg_eig  = np.sum(eigvals < 0)
180             n_small_eig = np.sum(eigvals < 1e-8)
181
182             print(f"   Min Eigenvalue:      {min_eigval:.4e}")
183             print(f"   Max Eigenvalue:      {max_eigval:.4e}")
184             print(f"   Negative Eigenvalues: {n_neg_eig}")
185             f"   {'<-- matrix is NOT positive semi-definite' if n_neg_eig > 0 else ''}")
186
187             print(f"   Near-Zero Eigenvals: {n_small_eig} (< 1e-8)")
188             f"   {'<-- rank deficient' if n_small_eig > 0 else ''}")
189
190             eigvals_pos = np.maximum(eigvals, 0)
191             explained    = np.cumsum(eigvals_pos[::-1]) / eigvals_pos.sum()
192             effective_rank = int(np.searchsorted(explained, 0.99)) + 1
193             print(f"   Effective Rank:      {effective_rank} / {n} (99% variance explained)")
194
195         except np.linalg.LinAlgError:

```

```

195         print(" Eigenvalue decomposition FAILED matrix is severely degenerate")
196
197     try:
198         cond = np.linalg.cond(cov)
199         if cond < 1e6:
200             cond_label = "Well-conditioned"
201         elif cond < 1e10:
202             cond_label = "Ill-conditioned optimization may be unstable"
203         else:
204             cond_label = "SEVERELY ill-conditioned optimization will likely fail"
205         print(f" Condition Number:      {cond:.4e} ({cond_label})")
206     except np.linalg.LinAlgError:
207         print(" Condition Number:      Could not be computed")
208         cond = np.inf
209
210     rank = np.linalg.matrix_rank(cov)
211     print(f" Matrix Rank:      {rank} / {n} {'<-- rank deficient' if rank < n
else ''}")
212
213     print()
214     print(" VERDICT:")
215     issues = []
216     if n_neg_var > 0: issues.append(f"      [!] {n_neg_var} assets have negative
variance")
217     if n_zero_var > 0: issues.append(f"      [!] {n_zero_var} assets have near-zero
variance")
218     if not is_symmetric: issues.append(f"      [!] Matrix is not symmetric")
219     if n_neg_eig > 0: issues.append(f"      [!] Not positive semi-definite Markowitz
will fail")
220     if cond > 1e10: issues.append(f"      [!] Condition number {cond:.1e} near-
singular")
221     if rank < n: issues.append(f"      [!] Rank deficient ({rank}/{n})")
222
223     for issue in issues:
224         print(issue)
225     if not issues:
226         print("      [OK] No critical issues detected")
227
228     def report(self) -> None:
229         """Skriver ut en komplett performancerapport."""
230         print("=" * 40)
231         print("PERFORMANCE REPORT")
232         print("=" * 40)
233         print(f"Cumulative Return:      {self.cumulative_return().iloc[-1]:.2%}")
234         print(f"Annualized Return:      {self.annualized_return():.2%}")
235         print(f"Annualized Volatility:{self.annualized_volatility():.2%}")
236         print(f"Sharpe Ratio:      {self.sharpe_ratio():.2f}")
237         print(f"Sortino Ratio:      {self.sortino_ratio():.2f}")
238         print(f"Max Drawdown:      {self.max_drawdown():.2%}")
239         print(f"Tracking Error:      {self.tracking_error():.2%}")
240         print(f"Information Ratio:      {self.information_ratio():.2f}")
241         print(f"Hit Rate:      {self.hit_rate():.2%}")
242         print(f"Win/Loss Ratio:      {self.win_loss_ratio():.2f}")
243
244         capm = self.capm_regression_metrics()
245         print(f"CAPM Alpha:      {capm['Alpha']:.4f}")
246         print(f"CAPM Beta:      {capm['Beta']:.4f}")
247         print(f"CAPM R:      {capm['r2']:.4f}")
248
249         print("\nFAMA-FRENCH FACTOR RESULTS")
250         print("-" * 40)

```

```

251     for factor, row in self.results.iterrows():
252         print(f"    Factor: {factor}")
253         print(f"        Mean Beta:    {row['mean_beta']:.4f}")
254         print(f"        Std Beta:      {row['std_beta']:.4f}")
255         print(f"        T-stat:        {row['t_stat']:.4f}")
256         print(f"        Significant: {row['significans']}")
257
258     print("\nCOVARIANCE MATRIX DIAGNOSTICS")
259     print("=" * 40)
260     self._report_cov_diagnostics()
261     print("=" * 40)
262
263     if self.betas_df is not None:
264         print(f"    LW Shrinkage Alpha:    {self.ledoit_wolf_shrinkage_intensity(self.
betas_df):.4f}")
265
266     @staticmethod
267     def performance_report_static(
268         port: pd.Series,
269         bench: pd.Series,
270         r: float,
271         label: str = "Portfolio"
272     ) -> None:
273         """
274         Fristående performancerapport utan klassinstans.
275         Används för att snabbt jämföra flera portföljserier,
276         t.ex. Markowitz vs. equal-weight i sektorbacktestingen.
277         """
278         common_index = port.index.intersection(bench.index)
279         port = port.loc[common_index]
280         bench = bench.loc[common_index]
281
282         if len(port) == 0:
283             print(f"\nIngen data för {label}")
284             return
285
286         excess_port = port - r
287         cum_return = (1 + port).prod() - 1
288         ann_return = (1 + cum_return) ** (12 / len(port)) - 1
289         ann_vol = port.std(ddof=1) * np.sqrt(12)
290         sharpe = (excess_port.mean() / port.std(ddof=1)) * np.sqrt(12) if port.std(
ddof=1) > 0 else np.nan
291
292         downside = np.minimum(excess_port, 0)
293         downside_dev = np.sqrt(np.mean(downside ** 2))
294         sortino = (excess_port.mean() / downside_dev) * np.sqrt(12) if downside_dev >
0 else np.nan
295
296         cum = (1 + port).cumprod()
297         max_dd = (cum / cum.cummax() - 1).min()
298
299         active = port - bench
300         te = active.std(ddof=1) * np.sqrt(12)
301         ir = (active.mean() / active.std(ddof=1)) * np.sqrt(12) if active.std(
ddof=1) > 0 else np.nan
302
303         excess_bench = bench - r
304         X = excess_bench.values.reshape(-1, 1)
305         y = excess_port.values
306         capm = LinearRegression()
307         capm.fit(X, y)

```

```

308     alpha = capm.intercept_ * 12
309     beta  = capm.coef_[0]
310     r2    = capm.score(X, y)
311
312     print(f"\nPERFORMANCE REPORT: {label}")
313     print("=" * 40)
314     print(f"Cumulative Return: {cum_return:.2%}")
315     print(f"Annualized Return: {ann_return:.2%}")
316     print(f"Annualized Vol: {ann_vol:.2%}")
317     print(f"Sharpe Ratio: {sharpe:.2f}")
318     print(f"Sortino Ratio: {sortino:.2f}")
319     print(f"Max Drawdown: {max_dd:.2%}")
320     print(f"Tracking Error: {te:.2%}")
321     print(f"Information Ratio: {ir:.2f}")
322     print(f"Periods: {len(port)}")
323     print(f"CAPM Alpha: {alpha:.2%}")
324     print(f"CAPM Beta: {beta:.2f}")
325     print(f"CAPM R^2: {r2:.2f}")

```

```

1  class SignalDiagnostics:
2      """
3      Faktor-nivå signalkvalitetsdiagnostik.
4      Bör köras på hela det rensade datasetet *innan* backtesting
5      för att utvärdera om faktorerna bär prediktiv information.
6      """
7
8      def information_coefficient(self, df: pd.DataFrame, factor_cols: list[str]) -> pd.
          DataFrame:
9          """
10             Beräknar Spearmans rang-korrelation (IC) mellan varje faktor
11             och Forward_return för varje månad.
12             """
13             records = []
14             df = df.copy()
15             df["Date"] = pd.to_datetime(df["Date"])
16
17             monthly_dates = (
18                 df.groupby(df["Date"].dt.to_period("M"))["Date"]
19                 .first().values
20             )
21
22             for date in monthly_dates:
23                 slice_ = df[df["Date"] == date].dropna(subset=["Forward_return"] + factor_cols
24                 )
25
26                 if len(slice_) < 10:
27                     continue
28                 row = {"Date": date}
29                 for col in factor_cols:
30                     x = pd.to_numeric(slice_[col], errors="coerce")
31                     y = pd.to_numeric(slice_["Forward_return"], errors="coerce")
32                     mask = x.notna() & y.notna()
33                     if mask.sum() < 5:
34                         row[col] = np.nan
35                         continue
36                     ic, _ = spearmanr(x[mask], y[mask])
37                     row[col] = ic
38                 records.append(row)
39
40             return pd.DataFrame(records).set_index("Date")
41
42     def ic_summary(self, ic_df: pd.DataFrame) -> pd.DataFrame:

```

```

41     """
42     Sammanfattar IC-serien: Mean IC, IC-std, ICIR och % positiva.
43     ICIR > 0.3 är typisk tröskel för en användbar faktor.
44     """
45     ic_numeric = ic_df.select_dtypes(include=[np.number])
46     return pd.DataFrame({
47         "Mean_IC":      ic_numeric.mean(),
48         "Std_IC":       ic_numeric.std(),
49         "ICIR":         ic_numeric.mean() / ic_numeric.std(),
50         "Pct_positive": (ic_numeric > 0).mean()
51     })
52
53 def signal_to_noise(self, betas_df: pd.DataFrame) -> pd.DataFrame:
54     """
55     Lambda SNR: mean(lambda_t) / std(lambda_t) per faktor.
56     Värde > 2.0 antyder att faktorpremien är urskiljbar från brus (~95%-nivå).
57     """
58     numeric = betas_df.select_dtypes(include=[np.number])
59     return pd.DataFrame({
60         "Mean": numeric.mean(),
61         "Std":  numeric.std(),
62         "SNR":  numeric.mean().abs() / numeric.std()
63     })
64
65 def factor_turnover(self, df: pd.DataFrame, factor_cols: list[str]) -> pd.DataFrame:
66     """
67     Mäter hur mycket tvärsektionella faktorränkningar förändras månad för månad.
68     Hög turnover = höga implicita transaktionskostnader.
69     """
70     df = df.copy()
71     df["Date"] = pd.to_datetime(df["Date"])
72     df["Month"] = df["Date"].dt.to_period("M")
73
74     monthly = (
75         df.groupby(["Month", "Instrument"])[factor_cols]
76         .last()
77         .reset_index()
78     )
79
80     records = {}
81     for col in factor_cols:
82         monthly["rank"] = monthly.groupby("Month")[col].rank(pct=True)
83         monthly_sorted = monthly.sort_values(["Instrument", "Month"])
84         diff = monthly_sorted.groupby("Instrument")["rank"].diff().abs()
85         records[col] = {"Mean_turnover": diff.mean(), "Std_turnover": diff.std()}
86
87     return pd.DataFrame(records).T
88
89 def report(self, df: pd.DataFrame, factor_cols: list[str], betas_df: pd.DataFrame) ->
pd.DataFrame:
90     """Skriver ut en komplett signalkvalitetsrapport."""
91     print("=" * 50)
92     print("SIGNAL DIAGNOSTICS REPORT")
93     print("=" * 50)
94
95     print("\n--- Signal-to-Noise Ratio (Lambda SNR) ---")
96     print(self.signal_to_noise(betas_df).to_string())
97
98     print("\n--- Information Coefficient Summary ---")
99     ic_df = self.information_coefficient(df, factor_cols)
100    print(self.ic_summary(ic_df).to_string())

```

```
101  
102     print("\n--- Factor Turnover ---")  
103     print(self.factor_turnover(df, factor_cols).to_string())  
104  
105     return ic_df
```